

Optimalisatie en uniformering van documentatie binnen Modern Workplace Infrastructuur bij ESC bv.

Wim Meirlaen.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. A. Veldeman

Co-promotor: Dhr. K De Jaeger

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Deze bachelorproef kwam tot stand vanuit een concreet praktijkprobleem binnen E.S.C. BV: het veilig en beheersbaar migreren van klantdocumentatie van Microsoft OneNote naar Remote Desktop Manager. Tijdens dit traject werd duidelijk hoe groot de impact van documentatiestructuur is op dagelijkse IT-operaties. Wat op het eerste gezicht een technische migratieopdracht lijkt, blijkt in de praktijk ook een oefening in governance, toegangsbeheer en kwaliteitscontrole.

Voor mij was dit onderzoek bijzonder leerrijk omdat het theorie en praktijk nauw samenbracht. De literatuur rond Privileged Access Management, information overload en beveiligingsprincipes bood een inhoudelijk kader, terwijl de implementatie met PowerShell en Microsoft Graph API die principes concreet toetsbaar maakte. De uitdaging lag niet alleen in het bouwen van scripts, maar vooral in het uitwerken van een robuuste workflow die ook op grotere schaal bruikbaar blijft.

Graag wil ik mijn promotor en co-promotor bedanken voor hun begeleiding, kritische feedback en inhoudelijke ondersteuning tijdens dit traject. Daarnaast bedank ik mijn collega's bij E.S.C. BV voor de praktische input en het vertrouwen om deze case in een realistische context uit te werken. Hun ervaring uit het werkveld heeft mee vorm gegeven aan de keuzes in deze bachelorproef.

Tot slot hoop ik dat dit werk niet alleen een bruikbaar resultaat oplevert voor de huidige migratiecontext, maar ook een basis vormt voor verdere professionalisering van documentatiebeheer en kennisdeling binnen de organisatie.

Samenvatting

IT-dienstverleners beheren voor honderden klanten gevoelige technische informatie: serveradressen, inloggegevens, netwerkconfiguraties en troubleshooting-documentatie. Bij E.S.C. BV groeide deze informatie historisch aan in Microsoft OneNote, verspreid over meerdere SharePoint-locaties per klant. Dit leidde tot structurele zoekproblemen, ontbrekende governance en aanzienlijke beveiligingsrisico's: OneNote had lange tijd geen ondersteuning voor sensitivity labels, hield niet bij wie credentials raadpleegde, en sloot daardoor niet aan bij de PAM-principes die vandaag nodig zijn. Wanneer één klant notitieboeken heeft op zowel de interne projectsite, de klantgerichte SharedWithCustomer-site als de centrale klantenhub, is tijdverlies bij het zoeken naar informatie onvermijdelijk.

E.S.C. BV stapte over naar Remote Desktop Manager (RDM) van Devolutions als centraal kennisplatform. Daarmee ontstond een concrete migratiebehoefte: hoe kan bestaande klantinformatie op een geautomatiseerde, veilige en schaalbare manier worden overgezet van OneNote naar RDM?

In dit onderzoek werden drie mogelijke extractiemethoden geëvalueerd. COM-automatisering via de OneNote Desktop-applicatie bleek onbetrouwbaar door blokkades van Windows-beveiligingsbeleid en antivirussoftware. Handmatige export is te tijdrovend bij honderden klantomgevingen. Microsoft Graph API vormt de meest haalbare aanpak, maar de vereiste `Notes.Read.All`-scope is op zichzelf breed geformuleerd. In deze implementatie werd dat correct afgebakend door uitsluitend gedelegeerde permissies te gebruiken en gebruikers enkel toegang te geven tot de relevante SharePoint-sites.

Als proof-of-concept werd een operationele batchpipeline uitgewerkt. Op orches-tratieniveau gebeuren export en import via `Bulk-Export-OneNote-Queue.ps1` en `Bulk-Import-Client-ToRDM2.ps1`. In de exportketen verwerkt `Export-OneNote-Deduplicated.ps1` via Microsoft Graph API alle pagina's, secties, afbeeldingen en bijlagen van de relevante SharePoint-sites per klant, met throttling-beheer (circuit breaker, exponentiële back-off en meergefasige herstelrondes). Een deduplicatiestap groepeerde pagina's op sectienaam en paginatitel en behoudt per groep de meest recente versie, zodat dubbele exports over meerdere sites niet onnodig worden geïmporteerd. In de importketen verwerkt `Import-ToRDM2.ps1` de geëxporteerde HTML-bestanden naar de juiste klantvault in RDM via de Devolutions.PowerShell-module. Daarbij wordt de tekstinhoud ook opgenomen in *Description* en *Documentation*, zodat inhoudszoekopdrachten bruikbaar blijven. Aanmelden gebeurt via een Entra ID-appregistratie met eenmalige code. Voor bulktoegang wordt

`Admin-Grant-SharedSites-From-CSV.ps1` gebruikt, zodat gebruikers alleen toegang krijgen tot sites die voor de export nodig zijn en de effectieve leesrechten beperkt blijven tot de toegewezen sites.

Ook op vlak van gebruiksgemak is de verbetering duidelijk. Voor de migratie moesten medewerkers eerst manueel in Teams lid worden van het juiste team (zoeken op `P-{klantnaam}`), daarna het `SharedWithCustomer`-kanaal zichtbaar maken en pas dan de juiste OneNote in de browser openen. Voor de collega's die exportbatches op de server uitvoeren, werd dit proces geautomatiseerd via de beheersscripts voor bulktoegang. Na import in RDM volstaat het om in de klantvault de map "OneNote export" te openen: de recente pagina's zijn al gededupliceerd en de informatie uit verschillende bronnotitieboeken is samengebracht in één consistente weergave.

De resultaten tonen aan dat geautomatiseerde migratie van OneNote naar RDM technisch haalbaar is. De voornaamste beperkingen liggen bij de brede formulering van `Notes.Read.All` op scope-niveau en de tijdsintensieve verwerking bij agressieve API-throttling. In de praktijk werd dit rechtenrisico beheerst via gedelegeerde permissies en gerichte sitetoekenning per gebruiker. AI-gestuurde categorisatie (zoals met Ollama) werd wel verkend als mogelijke vervolgstap, maar viel buiten de scope van deze bachelorproef.

Dit onderzoek biedt een praktische basis voor bedrijfsbrede uitrol binnen E.S.C. BV en is relevant voor IT-dienstverleners die staan voor gelijkaardige migratieuitdagingen vanuit ongestructureerde kennisplatformen naar gecentraliseerde, beveiligde informatiesystemen.

Inhoudsopgave

Lijst van figuren	ix
Lijst van tabellen	x
Lijst van codefragmenten	xi
Lijst van codefragmenten	xi
1 Inleiding	1
1.1 Probleemstelling	2
1.1.1 Gefragmenteerde informatieopslag	2
1.1.2 Beveiligings- en governancetekortkomingen	2
1.1.3 Structurele tekortkomingen in de huidige RDM-inrichting	3
1.1.4 Ontbreken van kant-en-klare migratiehulpmiddelen	4
1.2 Onderzoeksvraag	4
1.2.1 Deelvragen probleemdomein	4
1.2.2 Deelvragen oplossingsdomein	4
1.3 Onderzoeksdoelstelling	5
1.4 Opzet van deze bachelorproef	6
2 Stand van zaken	7
2.1 Privileged Access Management als primaire migratiedrijfveer	7
2.1.1 De PAM-aanpak	7
2.1.2 Credential vaults en secure repositories	8
2.1.3 Role-Based Access Control (RBAC)	8
2.1.4 Password vaulting en privileged session management	9
2.1.5 Audit trails en naleving	9
2.1.6 Praktijkvoorbeeld: Remote Desktop Manager	10
2.2 OneNote als kennismanagementsysteem	11
2.3 Sensitivity labeling in Microsoft 365	11
2.3.1 Het verschil tussen permissions en labeling	12
2.3.2 Risico's van ongelabelde OneNote notebooks	12
2.3.3 Remote Desktop Manager als veiliger alternatief	13
2.4 Microsoft Graph API voor OneNote-datamontage	13
2.4.1 Architectuur van de OneNote Graph API	13
2.4.2 Authenticatie en autorisatie	14
2.4.3 Throttling en API-limieten	15

2.4.4	Beperkingen voor grootschalige migraties	15
3	Methodologie	16
3.1	Fase 1: Probleemanalyse en inventarisatie	16
3.2	Fase 2: Evaluatie van extractiemethoden	17
3.2.1	Microsoft Graph API	17
3.2.2	COM-automatisering via OneNote Desktop	18
3.2.3	Handmatige export	18
3.2.4	Keuze: Microsoft Graph API	18
3.3	Fase 3: Authenticatieopzet in Microsoft Entra ID	19
3.3.1	Appregistratie: naam en tenant-scope	19
3.3.2	Public client flow inschakelen	19
3.3.3	API-permissies: gedelegeerd, niet als applicatie	19
3.3.4	Admin consent	20
3.3.5	Operationele scriptvolgorde	21
3.4	Fase 4: Ontwikkeling van de migratiescripts	22
3.4.1	Exportscript: Export-OneNote-Deduplicated.ps1	22
3.4.2	Boomstructuur en parent-entiteiten	24
3.4.3	Analysescript: Analyze-Hydrex-Full.ps1	25
3.4.4	Importsript: Bulk-Import-Client-ToRDM2.ps1	26
3.5	Fase 5: Productiegerichte batchpipeline en ontwerpkeuzes	28
3.5.1	Scriptstructuur en aanroepketen	28
3.5.2	Operationele logging en crash-diagnostiek	29
3.5.3	Fuzzy matching als kern-challenge	31
3.5.4	Onbewaakte batchverwerking en betrouwbaarheid	31
3.5.5	Praktische serveropzet voor langdurige batchruns	34
3.5.6	Data integrity en duplicate detection	36
3.5.7	API rate limiting en intelligent backoff	37
3.5.8	Configuration-driven architecture	39
3.5.9	Two-phase architecture (export + import)	41
3.5.10	HTML reconstruction en moeilijkheden met image magic bytes	41
3.5.11	Phased rollout en change management	44
3.5.12	Nazorg: OneNote-pagina's vervangen door RDM-verwijzing	45
3.6	Fase 6: Validatie en aanbevelingen	45
4	Conclusie	47
A	Onderzoeksvoorstel	49
A.1	Inleiding	49
A.1.1	Kadering en doelgroep	49
A.1.2	Probleemstelling	49
A.1.3	Onderzoeksvraag	49
A.1.4	Onderzoeksdoelstelling	50

A.2	Literatuurstudie	51
A.2.1	Knowledge Management en documentatiefragmentatie.	51
A.2.2	Best practices en technologieën.	51
A.2.3	DevOps voor documentatie.	51
A.2.4	Kennishiaat.	51
A.3	Methodologie	51
A.3.1	Fase 1: Inventarisatie en audit (1,5 weken).	52
A.3.2	Fase 2: Requirements-analyse (2,5 weken)	52
A.3.3	Fase 3: Vergelijkende studie (2 weken).	52
A.3.4	Fase 4: Ontwerp en ontwikkeling (4 weken)	52
A.3.5	Fase 5: Evaluatie en implementatieplan (2 weken).	53
A.3.6	Tijdsplanning overzicht.	53
A.4	Verwacht resultaat, conclusie	53
A.4.1	Verwachte bevindingen	53
A.4.2	Verwachte proof-of-concept	53
A.4.3	Meetbare verbeteringen	53
A.4.4	Meerwaarde	53

Bibliografie**55**

Lijst van figuren

3.1	Toegekende Entra-rechten voor de bulk-exportapp	20
3.2	Volledige operationele migratieflow	21
3.3	Boomstructuur van OneNote naar RDM	24
3.4	Vaultnaam als zoekterm	32
3.5	Procesflow onbewaakte export-run	33
3.6	Procesflow onbewaakte import-run	35
3.7	Duplicate-detectie in de exportqueue	38
3.8	Throttle-afhandeling	40
3.9	Configuratievalidatie bij opstart	42
3.10	Twee-fasen migratie via exportbestanden	43
3.11	Detectie via magic bytes	44

Lijst van tabellen

3.1	Operationele dagplanning migratiebatch	36
-----	--	----

Lijst van codefragmenten

3.1	Device code authenticatie in het exportscript	20
3.2	Opbouw van een deduplicatiesleutel per OneNote-pagina	23
3.3	Selectie van de nieuwste versie bij duplicaten	23
3.4	Wegschrijven van deduplicatie-metadata voor controle en audit	23
3.5	Afleiding van parentpad en kinddetectie voor het behoud van de boomstructuur	25
3.6	Keuze tussen folder-entry en document-entry op basis van onderliggende pagina's	25
3.7	Automatische voorselectie van de RDM-vault op basis van naamscore	27
3.8	Importaanroep in bulk-importscript	27
3.9	Zoektekst toevoegen aan importvelden	28
3.10	Vullen van de documentation-editor	28
3.11	Persistente logfile met niveaus	30
3.12	CSV-registratie van resterende fouten	30
3.13	Fuzzy matching voor vault-voorselectie	31
3.14	Quick Edit uitschakelen om onbewaakte runs robuust te maken	34
3.15	Duplicate-detectie en selectie van de nieuwste versie in de exportqueue	37
3.16	Throttle-afhandeling met cooldown in de export-queue	39
3.17	Config-gedreven startup met harde validatie van scripts en JSON-config	39
3.18	Van onbekend .bin-bestand naar correct type via magic bytes	43
3.19	Persistente opvolging van succesvol afgewerkte klanten voor gefaseerde uitrol	45

1

Inleiding

Effectieve kennisdeling en gestructureerd documentatiebeheer zijn cruciale pijlers voor de dagelijkse werking van IT-dienstverleners (Abbas et al., 2022). Wanneer klantinformatie verspreid is over meerdere systemen zonder duidelijke structuur of toegangsbeleid, leidt dit onvermijdelijk tot tijdverlies, kennisverliezen en verhoogde veiligheidsrisico's. Voor het Modern Workplace Infrastructure-team van E.S.C. BV is dit geen theoretisch vraagstuk, maar een concrete dagelijkse uitdaging. E.S.C. BV is een IT-dienstverlener die infrastructuurbeheer verzorgt voor een groot klantenbestand. Over de jaren groeide de technische documentatie per klant — netwerkschema's, credentials, serverconfiguraties, troubleshooting-handleidingen — organisch aan in Microsoft OneNote. Dit platform was toegankelijk en laagdrempelig, maar de informatie verspreidde zich over meerdere, slecht afgebakende SharePoint-locaties. Intussen stapte het bedrijf over naar Remote Desktop Manager (RDM) van Devolutions als centraal beheerplatform. Daarmee rijst de vraag: hoe kunnen honderden klantdossiers op een geautomatiseerde, betrouwbare manier worden gemigreerd van OneNote naar RDM?

Die vraag is in de praktijk belangrijker dan ze op het eerste gezicht lijkt. Voor een afzonderlijke klant lijkt het misschien haalbaar om informatie handmatig te herstructureren, maar op schaal groeit de complexiteit snel. Elke extra klant brengt opnieuw pagina's, bijlagen, historische notities en uitzonderingen mee. Zonder geautomatiseerde aanpak moet een medewerker telkens opnieuw bepalen waar informatie staat, welke pagina de recentste versie bevat en hoe die inhoud technisch correct naar RDM kan worden overgezet. Dat maakt migratie niet alleen traag, maar ook foutgevoelig. Een generieke oplossing moet daarom robuust genoeg zijn om met verschillen in structuur, paginavormgeving en bestandsbijlagen om te gaan.

Daarom werd dit onderzoek niet opgevat als een louter technisch exportproject, maar als een oefening in procesoptimalisatie. De centrale uitdaging bestond erin

om een reproduceerbare workflow te ontwerpen die voldoende veilig, voldoende snel en voldoende controleerbaar is om herhaaldelijk op verschillende klanten te worden toegepast. Dat betekent dat niet alleen de inhoud van de pagina's relevant is, maar ook de manier waarop de gegevens worden opgehaald, gevalideerd, hernoemd, gededupliceerd en uiteindelijk in RDM terechtkomen. De bachelorproef onderzoekt dus zowel het inhoudelijke doel als het operationele traject ernaartoe.

1.1. Probleemstelling

De doelgroep van dit onderzoek is het Modern Workplace Infrastructure-team van E.S.C. BV, bestaande uit consultants, support-medewerkers en projectleiders. De probleemstelling kent twee onderling verbonden dimensies.

1.1.1. Gefragmenteerde informatieopslag

Voor één klant kunnen notitieboeken zich op drie verschillende SharePoint-locaties bevinden:

1. **Interne projectsites:** documentatie onder `/sites/P-{klantnaam}` die enkel voor intern gebruik bestemd is
2. **SharedWithCustomer-sites:** klantgerichte documentatie onder `/sites/P-{klantnaam}-Sharedwithcustomer`, bedoeld voor gedeeld gebruik met de klant
3. **Klantspecifieke A-sites:** per klant een aparte SharePoint-site onder `/sites/{klantnummer}`, aangeduid met het prefix A- gevolgd door de klantnaam. Deze sites bevatten overkoepelende klantinformatie en zijn te herkennen aan het klantnummer in de URL.

Dit maakt het voor medewerkers tijdrovend en frustrerend om de juiste informatie terug te vinden. Voor zowel de gewone projectsites (`/sites/P-{klantnaam}`) als de SharedWithCustomer-sites geldt bovendien een extra drempel: medewerkers moeten eerst via Teams lid worden van het juiste klantteam voordat ze de bijhorende OneNote-notitieboeken kunnen openen. Nieuwe teamleden die worden ingewerkt voor een specifieke klant moeten op drie plaatsen zoeken, zonder garantie dat ze alles gevonden hebben (Tomsu, 2024). Het risico op dubbele, tegenstrijdige of verouderde informatie is reëel.

1.1.2. Beveiligings- en governancetekortkomingen

OneNote had lange tijd geen ondersteuning voor sensitivity labels (Hay, 2025). Voor een IT-dienstverlener die gevoelige klantcredentials beheert, gaf dat duidelijke veiligheidsrisico's:

- Credentials worden opgeslagen als platte tekst, zonder encryptie op record-niveau of toegangscontrole per item

- Er is geen duidelijke logging: SharePoint toont wie een pagina opende, maar niet wat er precies werd gelezen of gekopieerd
- Met de intrede van Microsoft 365 Copilot kan AI onbedoeld vertrouwelijke klantinformatie opvragen uit ongelabelde notitieboeken (Hay, 2025)
- Het principe van *least privilege* is onmogelijk te handhaven: wie toegang heeft tot een SharePoint-site, heeft toegang tot alle inhoud ervan (Garbis & Chapman, 2021)

In tegenstelling tot OneNote vertrekt Remote Desktop Manager van een meer gestructureerd model, waarbij informatie per klant in een vaste vaultstructuur wordt beheerd en gecombineerd wordt met toegangscontrole en logging (Devolutions, 2025). Die aanpak sluit aan bij PAM-principes zoals least privilege en auditability (Waltermire et al., 2018).

1.1.3. Structurele tekortkomingen in de huidige RDM-inrichting

Naast het migratievraagstuk kende ook de bestaande RDM-omgeving bij E.S.C. BV bij de start van dit onderzoek twee structurele problemen die de efficiëntie en veiligheid ondermijnden:

- **Bestaande klanten in een gedeelde default-vault (startsituatie):** bij de start van deze bachelorproef zaten bestaande klanten in één gedeelde default-vault. Dit creëerde prestatieproblemen bij een grote hoeveelheid entries en ondermijnde de veiligheid: medewerkers met toegang tot de default-vault zagen automatisch de gegevens van *alle* klanten daarin, wat ingaat tegen het least-privilege-principe (Garbis & Chapman, 2021). Tijdens het traject werden deze klanten stapsgewijs ondergebracht in individuele vaults, waardoor dit specifieke probleem intussen is weggewerkt.
- **Vault aanmaken voor nieuwe klanten:** niet elke medewerker beschikt over de rechten om een nieuwe vault aan te maken in RDM wanneer een nieuw klantproject start. Dit creëert een bewuste afhankelijkheid van een beperkt aantal beheerders die hiervoor gemachtigd zijn.

Een volledige oplossing vereiste dus niet alleen de migratie van bestaande OneNote-inhoud, maar ook een herstructurering van de RDM-omgeving zelf: de migratie van klanten uit de default-vault naar individuele vaults, en een beleid voor wie vaults mag aanmaken. De eerste component werd tijdens deze bachelorproef operationeel gerealiseerd; voor de tweede bepaalt de RDM-beheerder wie het recht krijgt om vaults aan te maken.

1.1.4. Ontbreken van kant-en-klare migratiehulpmiddelen

Er bestaan geen standaardtools die de overgang van OneNote naar RDM automatisch uitvoeren. De drie beschikbare extractiemethoden voor OneNote-gegevens (Microsoft Graph API, COM-automatisering en handmatige export) brengen elk eigen uitdagingen en beperkingen met zich mee, zoals besproken in Hoofdstuk 3. Dat gebrek aan kant-en-klare tooling verklaart waarom een proof-of-concept hier een meerwaarde heeft. De praktische realiteit is immers dat documentatie niet in één nette, uniforme vorm bestaat. Een migratietool moet omgaan met verspreide opslag, verschillende inhoudstypen en een organisatiecontext waarin rechten en toegangsbeheer net zo belangrijk zijn als de inhoud zelf. Voor E.S.C. BV is het doel daarom niet alleen om data te verplaatsen, maar om een werkbare basis te creëren voor een bredere administratieve standaardisering. De migratie naar RDM is in die zin ook een eerste stap naar een consequenter documentatiebeleid, waarin informatie beter vindbaar, beter beveiligd en beter onderhoudbaar wordt.

1.2. Onderzoeksvraag

De centrale onderzoeksvraag luidt:

Hoe kan klantinformatie op een geautomatiseerde, veilige en schaalbare manier worden gemigreerd van Microsoft OneNote naar Remote Desktop Manager bij E.S.C. BV, rekening houdend met de beperkingen van de beschikbare API's en de beveiligingsvereisten van de organisatie?

1.2.1. Deelvragen probleemdomein

- Welke structurele tekortkomingen heeft de huidige OneNote-gebaseerde kennisopslag voor gevoelige klantinformatie vanuit een veiligheids- en governanceperspectief?
- Op welke SharePoint-locaties is klantinformatie verspreid binnen de Microsoft 365-tenant van E.S.C. BV, en hoe kan dit geautomatiseerd in kaart worden gebracht?
- Welke beperkingen stelt Microsoft Graph API aan de extractie van OneNote-gegevens, en hoe verhouden deze zich tot de beveiligingsvereisten van de organisatie?

1.2.2. Deelvragen oplossingsdomein

- Hoe kan een PowerShell-exportscript worden ontwikkeld dat OneNote-notitieboeken via Microsoft Graph API uithaalt, inclusief afbeeldingen en bijlagen, met correcte omgang met throttling en API-limieten?
- Op welke manier kan geëxporteerde informatie worden ingeladen in de correcte klantvault in Remote Desktop Manager via de Devolutions PowerShell-

module?

- Hoe kan de authenticatie en autorisatie voor Microsoft Graph worden ingericht zodanig dat het migratieproces beheersbaar en beveiligd is?
- Welke benadering is aangewezen voor de automatische categorisatie van geïmporteerde OneNote-inhoud, rekening houdend met privacyvereisten voor klantdata?

1.3. Onderzoeksdoelstelling

Het beoogde resultaat van deze bachelorproef is een werkende proof-of-concept van een geautomatiseerde migratiepipeline van OneNote naar RDM. Concreet omvat dit:

- Een analyse van de beveiligingsproblemen en structurele tekortkomingen van OneNote als opslagplaats voor gevoelige klantinformatie bij IT-dienstverleners
- Een vergelijkende evaluatie van de beschikbare extractiemethoden voor OneNote-gegevens (MS Graph API, COM-automatisering, handmatige export)
- Een werkend PowerShell-exportscript dat notitieboekinhoud (tekst, afbeeldingen, bijlagen, subpagina's) extraheert via Microsoft Graph API
- Een werkend PowerShell-importscript dat de geëxporteerde gegevens in de juiste klantvault in RDM plaatst
- Een beschrijving van de vereiste authenticatieopzet in Microsoft Entra ID en de bijbehorende beveiligingsoverwegingen
- Een contextuele situering van de structurele tekortkomingen in de RDM-inrichting bij de start van het onderzoek (gedeelde default-vault en beperkte spreiding van vault-aanmaakrechten), met duidelijke grenzen van wat binnen deze bachelorproef is uitgewerkt
- Aanbevelingen voor verdere uitrol en het omgaan met de geïdentificeerde beperkingen

De bachelorproef is succesvol wanneer de proof-of-concept aantoont dat de migratie voor minstens één klantvault volledig en correct kan worden uitgevoerd, met behoud van de informatiestructuur, de hiërarchie van secties en subpagina's, en de gekoppelde mediabestanden. Daarnaast moet de oplossing aantonen dat bulk-migraties in grote aantallen stabiel en langdurig kunnen draaien zonder gebruikers-input tijdens de run.

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een literatuurstudie gepresenteerd over de relevante domeinen: beveiligingsrisico's van ongestructureerde kennisopslag, Privileged Access Management en credential management, de technische achtergrond van Microsoft Graph API voor OneNote, en Remote Desktop Manager als doelplatform.

In Hoofdstuk 3 wordt de methodologie toegelicht: de gefaseerde aanpak, de evaluatie van extractiemethoden, de authenticatieopzet in Entra ID en de ontwikkeling van de migratiescripts.

In Hoofdstuk 4, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek.

De opbouw van deze bachelorproef is dus bewust praktijkgericht. Eerst wordt het probleem vanuit de praktijk gedefinieerd, daarna wordt de stand van zaken in het domein onderzocht, vervolgens wordt de gekozen methode stap voor stap verantwoord en ten slotte worden de resultaten kritisch samengevat. Die volgorde is relevant omdat de technische oplossing pas zinvol is wanneer ze in de context van de organisatie wordt bekeken. Een migratiescript dat op zichzelf werkt, maar de structuur, rechten of snelheid van de organisatie niet respecteert, is immers geen duurzame oplossing.

2

Stand van zaken

Dit hoofdstuk bevat de literatuurstudie over kennismigratie, information overload en de rol van AI bij het beheer van bedrijfsinformatie. De inhoud bouwt voort op de inleiding en vormt de wetenschappelijke basis voor een geautomatiseerde migratie van OneNote naar RDM. Eerst wordt de belangrijkste migratiereden uitgewerkt vanuit PAM.

2.1. Privileged Access Management als primaire migratie-drijfveer

Voor IT-dienstverleners die toegang beheren tot gevoelige klantomgevingen is Privileged Access Management (PAM) essentieel. PAM-systemen beheren bevoorrechte accounts en credentials op een gestructureerde manier, met beveiliging als basis en niet als toevoeging achteraf.

2.1.1. De PAM-aanpak

Privileged Access Management omvat het beheer en de beveiliging van accounts met verhoogde privileges binnen IT-systemen (Garbis & Chapman, [2021](#), pp. 155–161). In tegenstelling tot reguliere gebruikersaccounts hebben bevoorrechte accounts toegang tot kritieke systemen, configuraties en gevoelige data. Sizov ([2024](#)) benadrukt dat inadequaet beheer van deze credentials een van de belangrijkste oorzaken is van moderne beveiligingsincidenten.

Bij traditionele benaderingen, zoals het opslaan van credentials in OneNote, ontbreekt de gestructureerde beveiliging die PAM-principes voorschrijven. Wachtwoorden worden bewaard als platte tekst, er is geen logging van toegang, en er zijn geen mechanismen om credential misbruik te detecteren of te voorkomen.

2.1.2. Credential vaults en secure repositories

Een kerncomponent van PAM is de credential vault: een beveiligde repository waarin bevoorrechte credentials worden opgeslagen (Garbis & Chapman, 2021, pp. 155–161). In plaats van dat individuele medewerkers wachtwoorden kennen en beheren, worden deze centraal opgeslagen met strikte toegangscontroles.

Volgens Waltermire et al. (2018) bieden credential vaults verschillende beveiligingsvoordelen:

- **Centralisatie:** Alle credentials op één beveiligde locatie
- **Encryptie:** Credentials worden versleuteld opgeslagen (at rest en in transit)
- **Toegangscontrole:** Fijnmazige permissies bepalen wie welke credentials mag ophalen
- **Rotatie:** Automatische wijziging van wachtwoorden volgens beleid
- **Auditing:** Volledige logging van wie wanneer welke credentials heeft opgevraagd

Dit verschilt sterk van OneNote, waar credentials vaak als tekst tussen andere notities staan, zonder fijnmazige toegangscontrole of duidelijke audit trail.

2.1.3. Role-Based Access Control (RBAC)

Role-Based Access Control is een fundamenteel principe in moderne toegangsbeheersystemen waarbij permissies worden toegekend op basis van rollen binnen een organisatie, niet aan individuele gebruikers (Blobel et al., 2006, pp. 597–623). Een medewerker krijgt een rol toegewezen (bijvoorbeeld “Junior Technicus Klant A” of “Senior Engineer Infrastructuur”), en die rol bepaalt welke systemen en gegevens toegankelijk zijn.

Garbis en Chapman (2021, pp. 155–161) beschrijven hoe RBAC in PAM-context de volgende voordelen biedt:

Least-privilegeprincipe

Gebruikers krijgen alleen de minimale toegangsrechten die nodig zijn voor hun functie. Een technicus die werkt aan de IT-omgeving van Klant A hoeft geen toegang te hebben tot credentials van Klant B. Dit beperkt de impact van een eventueel beveiligingsincident.

Scheiding van taken

Kritieke handelingen vereisen meerdere rollen. Bijvoorbeeld: een technicus kan een wijziging voorbereiden, maar een senior engineer moet deze goedkeuren voordat deze wordt uitgevoerd. In OneNote is een dergelijke scheiding onmogelijk te implementeren.

Schaalbaarheid

Bij aanneme van een nieuwe medewerker wordt deze toegewezen aan bestaande rollen. Bij vertrek wordt de rol ingetrokken, en automatisch verliest de persoon toegang tot alle bijbehorende credentials en systemen. In een OneNote-gebaseerde omgeving kan toegang ook via Teams- of SharePoint-groepen worden ingetrokken, maar die werkt meestal op site- of notebookniveau en minder fijnmazig op credential- of itemniveau. Daardoor blijft het risico groter dat gebruikers tijdens hun actieve toegangsperiode breder toegang krijgen dan operationeel nodig is; bij intrekking van die rechten vervalt die toegang uiteraard opnieuw.

2.1.4. Password vaulting en privileged session management

Garbis en Chapman (2021) leggen PAM in deze context vooral uit met twee praktische bouwstenen: *password vaulting* (Garbis & Chapman, 2021, pp. 155–156) en *privileged session management* (Garbis & Chapman, 2021, pp. 157–158). Bij password vaulting staan credentials centraal in een beveiligde kluis. Toegang verloopt gecontroleerd via aanvraag en goedkeuring (“checking out”). Wachtwoorden kunnen periodiek roteren wanneer die functie in het PAM-platform is geconfigureerd. In sommige gevallen logt het PAM-systeem de gebruiker automatisch in, zonder het wachtwoord te tonen. One-time passwords (OTP) versterken de aanmelding, maar zijn niet hetzelfde als de rotatie van het onderliggende accountwachtwoord. Bij privileged session management maakt een beheerder geen rechtstreekse verbinding meer met een server of systeem. De verbinding loopt via het PAM-systeem als tussenstap. Dat maakt het mogelijk om elke beheersessie op te volgen, te loggen en indien nodig te onderbreken.

Dit biedt kritieke beveiligingsvoordelen:

- Gebruikers kunnen credentials niet (per ongeluk) delen of opschrijven
- Indien rotatie actief is, kunnen wachtwoorden automatisch worden gewijzigd zonder gebruikers te belasten
- Bij vertrek van een medewerker hoeven wachtwoorden niet te worden gereset, alleen de toegangsrechten worden ingetrokken
- Sessies kunnen worden opgenomen voor naleving van regels en forensische analyse

Sizov (2024) benadrukt dat deze aanpak fundamenteel veiliger is dan het traditionele model waarbij credentials worden gedeeld en medewerkers deze kennen.

2.1.5. Audit trails en naleving

PAM-systemen bieden uitgebreide logging en auditing die belangrijk zijn voor naleving van regelgeving zoals AVG/GDPR, NIS2 en branchespecifieke standaar-

den (Waltermire et al., 2018). Elke interactie met bevoorrechte accounts wordt gelogd:

- Wie heeft welke credential opgevraagd?
- Wanneer en hoe lang was een sessie actief?
- Welke acties zijn uitgevoerd tijdens de sessie?
- Zijn er afwijkende patronen detecteerbaar (bijvoorbeeld toegang op ongebruikelijke tijden)?

Volgens Blobel et al. (2006, pp. 597–623) is zo'n audit trail niet alleen nuttig voor naleving, maar ook voor forensisch onderzoek en het opsporen van interne dreigingen. In OneNote is een dergelijke audit trail volledig afwezig. SharePoint logt weliswaar wie een pagina heeft geopend, maar niet wat er is gelezen, gekopieerd of verder verspreid. Dit maakt het nagenoeg onmogelijk om te traceren wie toegang heeft gehad tot specifieke klantcredentials.

2.1.6. Praktijkvoorbeeld: Remote Desktop Manager

Tools zoals Remote Desktop Manager illustreren hoe deze PAM-principes in de praktijk kunnen worden toegepast voor IT-dienstverleners (Devolutions, 2025). Dergelijke systemen implementeren credential vaults, RBAC, password vaulting, session recording en audit trails in een geïntegreerd platform, in lijn met de beschreven PAM-principes (Waltermire et al., 2018). Voor organisaties zoals E.S.C. BV die overwegen te migreren van OneNote naar een gestructureerder systeem, demonstreren zulke tools de praktische haalbaarheid van het toepassen van PAM-principes op dagelijkse IT-operaties.

De belangrijkste meerwaarde zit niet in één tool, maar in de onderliggende principes: credentials gestructureerd beheren, toegang fijn afbakenen en veiligheid van bij het ontwerp meenemen. Die principes gelden zowel voor Remote Desktop Manager en gelijkaardige platformen voor klantdocumentatie als voor gespecialiseerde PAM-tools.

Voor E.S.C. BV is net die bredere kijk belangrijk. De organisatie beheert niet alleen technische accounts, maar ook operationele kennis met een gelijkaardige impact. Een foutieve configuratiehandleiding, een oud wachtwoord of een onvolledige herstelprocedure kan in de praktijk even schadelijk zijn als een gedeelde login. Daarom is het logisch om documentatie niet los van PAM te behandelen, maar als deel ervan. Kennis is in deze context geen losse notitie, maar bedrijfskritische informatie die dezelfde discipline vraagt als andere gevoelige data.

Daarom is de migratie van OneNote naar RDM meer dan een cosmetische herschikking. OneNote laat veel vrijheid toe, maar op schaal werkt die vrijheid vaak tegen de organisatie. Informatie wordt snel opgeslagen, maar niet altijd op een herbruikbare of controleerbare manier. RDM dwingt meer structuur af, zodat gegevens

meteen in een bruikbare categorie terechtkomen. Net dat maakt de keuze voor RDM in deze bachelorproef inhoudelijk relevant, en niet enkel praktisch handig.

2.2. OneNote als kennismanagementsysteem

Microsoft OneNote is een veelgebruikte tool voor notities en samenwerking binnen organisaties. Het platform wordt gebruikt voor onder meer probleemoplossingsgidsen, evaluaties na incidenten en teamdocumentatie (Hay, 2025). Die flexibiliteit maakt OneNote populair, maar zorgt ook voor uitdagingen rond informatiebeveiliging en gestructureerd beheer.

Een belangrijk kenmerk van OneNote is dat notebooks zowel persoonlijk als gedeeld gebruikt kunnen worden. Die veelzijdigheid kan leiden tot ongecontroleerde verspreiding van gevoelige informatie. Hay (2025) wijzen erop dat “notebook oversharing” een groot probleem is geworden, waarbij vertrouwelijke bedrijfsinformatie onbedoeld toegankelijk wordt voor onbevoegde gebruikers.

Met de introductie van AI-assistenten zoals Microsoft 365 Copilot is dit probleem groter geworden. In het AI-tijdperk volstaat het niet meer om te rekenen op verborgen of versnipperde informatie: AI maakt het makkelijk om toegankelijke informatie te doorzoeken en tonen, ongeacht de locatie. Dit maakt het des te belangrijker om kennismanagement te structureren volgens duidelijke categorieën en toegangsrechten (Hay, 2025).

Deze beveiligings- en structureringsproblemen zijn een belangrijke reden om te migreren naar meer gestructureerde systemen zoals RDM, waar informatie vanaf het begin gecategoriseerd en beveiligd wordt.

In de praktijk heeft die verschuiving ook een menselijk voordeel. Wanneer informatie verspreid staat over losse notities, blijft kennis sterk persoonsgebonden: alleen wie de voorgeschiedenis kent, weet waar iets staat en waarom het op die plaats is beland. Een gestructureerd systeem verlaagt die afhankelijkheid van individuele medewerkers. Nieuwe teamleden kunnen sneller instappen omdat de informatie niet alleen beschikbaar is, maar ook logisch geordend is. Daarmee raakt de problematiek van OneNote niet alleen beveiliging en naleving, maar ook continuïteit van dienstverlening en overdraagbaarheid van kennis.

2.3. Sensitivity labeling in Microsoft 365

Microsoft heeft sensitivity labeling ingevoerd als belangrijke beveiligingsmaatregel voor data in Microsoft 365. Sensitivity labels bieden encryptie en beleids-handhaving, zodat content beschermd blijft, ook wanneer ze wordt verplaatst of gedeeld (Hay, 2025). Dat verschilt van traditionele permission-based security, die vooral bepaalt wie op een bepaald moment toegang heeft.

2.3.1. Het verschil tussen permissions en labeling

Permission-based beveiliging, zoals in SharePoint- en Teams-sites, regelt toegang op basis van rechten binnen een specifieke situatie. Wie permissie heeft op een SharePoint-site, kan in principe alle content op die site bekijken, inclusief OneNote-notebooks. In het AI-tijdperk heeft die aanpak duidelijke beperkingen.

Sensitivity labeling voegt daarentegen een beveiligingslaag toe die met het document meereist. Hay (2025) legt uit dat labeling bepaalt welke confidentialiteits- en beveiligingseisen AI-tools zoals Copilot moeten respecteren. Labels helpen gebruikers ook om de gevoeligheid van content te begrijpen en ervoor te zorgen dat gegenereerde AI-responses met de gepaste voorzichtigheid worden behandeld.

2.3.2. Risico's van ongelabelde OneNote notebooks

OneNote was tot voor kort de enige Microsoft 365-applicatie zonder ondersteuning voor sensitivity labels (Hay, 2025). Voor organisaties zoals E.S.C. BV, waar OneNote-notebooks verspreid staan over verschillende SharePoint-pagina's en Teams-sites zonder labeling, geeft dat belangrijke beveiligingsrisico's:

Onbedoelde blootstelling van data door AI

Microsoft 365 Copilot kan alle informatie doorzoeken waartoe een gebruiker toegang heeft, ongeacht de locatie. Zonder sensitivity labels kan Copilot vertrouwelijke klintinformatie uit OneNote-notebooks oppikken en gebruiken in AI-antwoorden, zelfs wanneer de gebruiker niet wist dat die informatie bestond. Dit fenomeen wordt door Hay (2025) geïllustreerd met de metafoor van een “vergiftigde appeltaart”: gebruikers moeten geïnformeerd worden wanneer Copilot zeer vertrouwelijke content gebruikt om een antwoord te genereren.

Onbedoeld delen van gevoelige klantdata

In een omgeving met meerdere klanten, elk met een eigen SharePoint-site, kunnen medewerkers onbedoeld toegang krijgen tot notebooks van andere klanten. Zonder expliciete labeling is niet meteen duidelijk welke informatie strikt vertrouwelijk is. Dit verhoogt het risico op onbedoelde verspreiding van gevoelige IT-configuraties, credentials of incidentrapporten.

Gebrek aan beleidshandhaving

Ongelabelde content kan niet worden afgedwongen via geautomatiseerd beveiligingsbeleid zoals data loss prevention (DLP), automatische versleuteling of beperkingen op extern delen. Dit betekent dat gevoelige klintinformatie mogelijk per ongeluk naar externe partijen kan worden verstuurd of op onveilige locaties kan worden opgeslagen.

Uitdagingen rond naleving en controle

Voor IT-dienstverleners is het essentieel om aan te tonen dat klantdata volgens de geldende richtlijnen wordt beheerd. Zonder labeling is het moeilijk om te traceren welke informatie als vertrouwelijk moet worden beschouwd en of deze volgens de juiste procedures is behandeld.

2.3.3. Remote Desktop Manager als veiliger alternatief

Op basis van productdocumentatie en PAM-richtlijnen kan RDM worden gezien als een meer gecontroleerde omgeving voor het beheren van klantinformatie dan een klassieke OneNote-opzet (Devolutions, 2025; Waltermire et al., 2018). De kern ligt minder bij de toolnaam zelf en meer bij het werkmodel dat de tool afdwingt: informatie wordt gestructureerd opgeslagen, rechten worden fijnmaziger beheerd, en handelingen worden beter traceerbaar.

Concreet betekent dit dat teams niet enkel sneller informatie terugvinden, maar ook beter kunnen aantonen wie toegang had tot welke gegevens en op welk moment. Dat is relevant voor operationele veiligheid en voor naleving van regels rond klantdata.

Voor deze bachelorproef is vooral het verschil in beheersdiscipline belangrijk. In OneNote kan informatie gemakkelijk groeien zonder uniforme indeling. In een RDM-achtige vaultstructuur wordt al bij het opslaan gekozen onder welke klant, map en context een item valt. Daardoor wordt toegang gericht toegekend en wordt de kans kleiner dat gevoelige informatie onbedoeld zichtbaar is voor een te brede groep.

In een AI-context blijft dat verschil ook relevant: hoe beter informatie vooraf is afgebakend via rollen en mappenstructuur, hoe kleiner de kans dat AI-antwoorden gebaseerd zijn op data buiten de bedoelde scope van de gebruiker. Daarom kan de overstap van OneNote naar RDM in deze studie inhoudelijk worden beschouwd als een beveiligingsverbetering, niet louter als een gebruiks- of layoutwijziging.

2.4. Microsoft Graph API voor OneNote-datamontage

Microsoft Graph API is de centrale REST-API voor Microsoft 365-resources, waaronder OneNote-notitieboeken (Microsoft, 2025d). Via Graph kunnen applicaties notitieboeken, secties en pagina's ophalen, inclusief de HTML-inhoud van elke pagina en de bijbehorende mediaresources zoals afbeeldingen en bijlagen.

2.4.1. Architectuur van de OneNote Graph API

Via de OneNote Graph API zijn vier soorten objecten beschikbaar: notitieboeken, secties, pagina's en mediabestanden. Het startpunt is `/sites/{site-id}/onenote/` (Microsoft, 2025d). De belangrijkste eindpunten zijn:

- `/notebooks`: alle notitieboeken op een site

- `/sections`: secties binnen een notitieboek
- `/pages`: pagina's binnen een sectie
- `/pages/{id}/content`: de inhoud van een pagina als HTML
- `/pages/{id}/resources/{resourceId}/content`: afbeeldingen en bijlagen bij een pagina

Een pagina lijkt in OneNote eenvoudig, maar bestaat achter de schermen uit meerdere onderdelen: de HTML-tekst, afbeeldingen en bijlagen die elk afzonderlijk moeten worden opgehaald (Microsoft, 2025d). Bij notitieboeken met veel media leidt dat tot een groot aantal afzonderlijke aanvragen naar de API.

De API levert de ruwe inhoud, maar bepaalt zelf niet waar die in RDM terechtkomt. Dat moet het exportscript zelf regelen: elke pagina moet op de juiste plaats in de juiste klantvault worden geplaatst. Daar zit de kern van het automatiseringswerk in deze bachelorproef.

2.4.2. Authenticatie en autorisatie

Voor toegang tot de OneNote Graph API wordt de appregistratie aangemeld via Microsoft Entra ID (voorheen Azure Active Directory) en krijgt deze de nodige toegangsrechten (Microsoft, 2025c). In dit opzet loopt de toegang via gedelegeerde Graph-rechten: de app mag OneNote-inhoud lezen namens de uitvoerende gebruiker, maar die gebruiker krijgt daardoor niet automatisch toegang tot de volledige tenant. In de praktijk werd de toegang bovendien vooraf beperkt tot de relevante SharePoint-sites via `Admin-Grant-SharedSites-From-CSV.ps1`. Het *least privilege*-principe blijft daarbij het uitgangspunt (Garbis & Chapman, 2021): de effectieve leesrechten zijn gekoppeld aan de sites waarvoor expliciet toegang is toegerekend.

Rechten van app en gebruiker

Microsoft Graph maakt een onderscheid tussen rechten die aan de app gekoppeld zijn en de toegang van de aangemelde gebruiker (Microsoft, 2025c). In deze bachelorproef werkt de app namens de gebruiker, maar de effectieve leesruimte blijft beperkt tot de SharePoint-sites waarvoor die gebruiker via `Admin-Grant-SharedSites-From-CSV.ps1` toegang kreeg. Daardoor kan de oplossing notitieboeken uitlezen binnen de toegewezen sites, zonder dat er sprake is van vrije toegang tot de volledige tenant.

Aanmelding met code

Voor scriptomgevingen zonder interactieve webinterface volstaat aanmelding met een eenmalige code (Microsoft, 2025b). De gebruiker bevestigt de toegang zelf in de browser en het script krijgt daarna tijdelijk toegang namens die gebruiker. Voor

deze bachelorproef is vooral dat functionele mechanisme relevant; de technische details van de aanmeldflow zijn hier minder belangrijk.

2.4.3. Throttling en API-limieten

Microsoft Graph API past throttling toe om overbelasting van de servers te voorkomen (Microsoft, 2025a). Wanneer een toepassing te veel aanvragen doet binnen een bepaalde tijdspanne, retourneert de API een HTTP 429-statuscode (*Too Many Requests*) met een `Retry-After`-header die aangeeft na hoeveel seconden opnieuw geprobeerd mag worden.

Voor OneNote-migraties waarbij honderden pagina's en duizenden mediaresources sequentieel worden opgehaald, is throttling een belangrijk knelpunt. Een robuuste implementatie vereist:

- **Adaptief vertragsingsbeheer:** de vertraging tussen aanvragen dynamisch aanpassen op basis van throttle-responses
- **Circuit breaker-patroon:** na een reeks opeenvolgende throttle-responses tijdelijk stoppen met aanvragen om de server te ontlasten
- **Exponentiële back-off:** de wachttijd na elke throttle-response stapsgewijs verhogen
- **Meerfasige herstelrondes:** mislukte aanvragen in meerdere rondes opnieuw proberen

2.4.4. Beperkingen voor grootschalige migraties

Naast de brede rechte set en throttling heeft de Graph OneNote API nog andere beperkingen die relevant zijn voor grootschalige migraties:

Paginerings

Grote notitieboeken worden niet in één keer teruggegeven, maar in kleinere delen. Daardoor moet de migratie de inhoud stap voor stap ophalen en verwerken, wat de uitvoering trager maakt (Microsoft, 2025d).

Geen bulk content-ophaling

De inhoud van pagina's moet afzonderlijk worden opgehaald; er is geen mogelijkheid om meerdere pagina's tegelijk in één keer te verwerken. Ook dat maakt de migratie noodzakelijkerwijs opeenvolgend (Microsoft, 2025d).

Tijdelijke resource-URLs

Media en bijlagen worden via tijdelijke verwijzingen opgehaald. Bij lange migraties moet de toegang daarom op tijd worden vernieuwd om onderbrekingen te vermijden (Microsoft, 2025d).

3

Methodologie

Dit onderzoek heeft als doel een geautomatiseerde oplossing te ontwikkelen voor de migratie van klantinformatie van Microsoft OneNote naar Remote Desktop Manager (RDM) van Devolutions binnen E.S.C. BV. De aanpak bestaat erin een concrete oplossing te bouwen, die daarna ook te testen en te verbeteren, zodat een echt bedrijfsprobleem wordt opgelost (Walker, [2007](#)). Het onderzoek verliep in vijf hoofdfasen.

3.1. Fase 1: Probleemanalyse en inventarisatie

De eerste fase bestond uit een gecombineerde literatuurstudie en situatie-inventarisatie. Enerzijds werd de wetenschappelijke en technische literatuur verkend rond information overload bij IT-specialisten (Tomsu, [2024](#)), beveiligingsproblemen van ongestructureerde kennisopslag (Hay, [2025](#)), en Privileged Access Management (Garbis & Chapman, [2021](#)). Anderzijds werd de concrete situatie bij E.S.C. BV in kaart gebracht:

- Inventarisatie van de SharePoint-structuur: welke types locaties worden gebruikt voor klantdocumentatie (projectsites, SharedWithCustomer-sites, klantenhub)?
- Analyse van de notitieboekstructuur per klant: secties, subpagina's, inhouds-types
- Identificatie van de informatiecategorieën die relevant zijn voor RDM (credentials, documentatie, netwerkinformatie, enz.)
- Bespreking met teamleden over knelpunten in het zoeken naar informatie en de gewenste doelstructuur in RDM

Uit deze fase bleek dat de informatie voor één klant verspreid kan staan over meerdere SharePoint-sites, en dat de notitieboeken tekst, tabellen, afbeeldingen en bijlagen bevatten die allemaal correct gemigreerd moeten worden.

Deze inventarisatie leverde drie directe ontwerpkeuzes op voor de migratie van OneNote naar RDM. Ten eerste moest de export meerdere sites per klant kunnen meenemen, omdat informatie vaak verspreid stond over projectsites en SharedWithCustomer-sites. Ten tweede moest de pipeline robuust omgaan met verschillen tussen klanten (meerdere notebooks, afwijkende naamgeving, overlap tussen interne en gedeelde documentatie) zonder manuele correcties per klant. Ten derde moest de oplossing herhaalbaar zijn voor bulkgebruik: niet enkel een geslaagde export voor één testklant, maar een consistente aanpak die voor meerdere klanten na elkaar werkt in productie.

Deliverable: Een concreet beeld van de bronstructuur en de eisen waaraan de pipeline moet voldoen: multi-site export per klant, ondersteuning voor alle inhoudstypes (tekst, afbeeldingen, bijlagen, subpagina's), deduplicatie, throttlingbestendigheid, herhaalbaarheid zonder handmatige tussenkomst, gecontroleerde toegang en traceerbaarheid via logbestanden.

3.2. Fase 2: Evaluatie van extractiemethoden

Voordat scripts werden ontwikkeld, werden de beschikbare methoden om data uit OneNote te extraheren systematisch geëvalueerd.

3.2.1. Microsoft Graph API

Microsoft Graph API biedt programmatische toegang tot OneNote-notitieboeken via REST-endpoints (Microsoft, 2025d). Voor deze aanpak is `Notes.Read.All` als gedelegeerde scope nodig. De precieze afbakening van rechten en de bijhorende veiligheidsmaatregelen worden gebundeld in [deelparagraaf 3.3.3](#).

Voordelen van Microsoft Graph

De Graph API is een aantrekkelijke keuze voor OneNote-extractie. Met een solide REST-endpoint (wel gedocumenteerd en ondersteund door Microsoft) kan een script notitieboeken, secties en pagina's ophalen. De API ondersteunt alle inhoudstypes: platte tekst, opmaak (bold, italic, enz.), tabellen, afbeeldingen en bijlagen komen allemaal via Graph beschikbaar. Bovendien hoeft op de server waarop het script draait geen actieve OneNote Desktop-applicatie geïnstalleerd te zijn; enkel een moderne PowerShell met internet en OAuth2-authenticatie volstaan. Dit maakt Graph schaalbaar en geschikt voor serveromgevingen.

Nadelen van Microsoft Graph

Echter, Graph kent ook duidelijke beperkingen. Ten eerste zijn de benodigde permissies (`Notes.Read.All`) vrij breed: het script mag alle notitieboeken van de aan-

gemelde gebruiker uitlezen, niet enkel de klantgerelateerde. Dit vereist strikte afbakening van gebruikersrechten op SharePoint-niveau. Ten tweede past Microsoft Graph agressieve throttling toe wanneer veel aanvragen snel volgen elkaar op. Voor grootschalige migraties van honderden pagina's en duizenden mediabestanden leidt dit tot vertraging en herpogingen. Tot slot moet voor elementen als afbeeldingen en bijlagen elk bestand afzonderlijk worden opgehaald via een aparte HTTP-aanroep: een pagina met tien afbeeldingen vereist dus tien extra requests naast de aanvraag voor de pagina-inhoud zelf. Dit cumulatieve effect maakt grote migraties relatief traag.

3.2.2. COM-automatisering via OneNote Desktop

Een alternatieve methode maakt gebruik van de COM-interface van de OneNote Desktop-applicatie. Via PowerShell kan een script de geopende notitieboeken in de OneNote-app uitlezen zonder netwerkpermissies.

Bij tests in de E.S.C. BV-omgeving bleek deze methode echter onbetrouwbaar. Het script kon geen geopend notitieboek detecteren, ondanks pogingen met Office-herstelmodule, herstart van OneNote en herstart van het systeem. Mogelijke oorzaken zijn Windows-beveiligingsbeleid dat COM-automatisering blokkeert, antivirussoftware die COM-aanroepen onderschept, of een beschadigde COM-registratie die niet wordt hersteld door de Office-herstelmodule.

3.2.3. Handmatige export

OneNote biedt ingebouwde exportfunctionaliteit per pagina of sectie. Voor een migratie van honderden klantdossiers met duizenden pagina's is dit echter te tijdrovend en foutgevoelig.

3.2.4. Keuze: Microsoft Graph API

Op basis van de evaluatie werd Microsoft Graph API geselecteerd als extractiemethode. De methode is technisch betrouwbaar en schaalbaar, maar vraagt een strikte rechtenafbakening. Daarom werd een expliciet rechtenmodel uitgewerkt met gedelegeerde permissies en gecontroleerde SharePoint-toegang per gebruiker (zie [deelparagraaf 3.3.3](#)).

Bij de keuze speelde ook mee of de oplossing in de praktijk goed bruikbaar was. Ze moest niet alleen werken, maar ook makkelijk opnieuw gebruikt kunnen worden door collega's. Graph API voldoet daaraan, omdat de aanpak goed te documenteren, te herhalen en te controleren is. Daarom bleef het, ondanks de beperkingen, de beste keuze.

3.3. Fase 3: Authenticatieopzet in Microsoft Entra ID

Voordat het exportscript kon worden gebruikt, moest een appregistratie worden aangemaakt in het Microsoft Entra ID-portaal (Microsoft, 2025c). Dit was nodig omdat de Microsoft Graph API OAuth2-authenticatie vereist: het script kan niet rechtstreeks met een gebruikersnaam en wachtwoord inloggen, maar moet via een geregistreerde applicatie een access token aanvragen.

3.3.1. Appregistratie: naam en tenant-scope

De applicatie werd geregistreerd onder de naam *OneNote-Bulk-Export* als *single tenant*-applicatie. De keuze voor single tenant betekent dat de applicatie uitsluitend gebruikt kan worden door accounts binnen de eigen Microsoft 365-tenant van E.S.C. BV. Een multi-tenant configuratie zou de applicatie ook toegankelijk maken voor externe organisaties, wat voor een intern migratiehulpmiddel een onnodig veiligheidsrisico zou vormen.

3.3.2. Public client flow inschakelen

De optie *Allow public client flows* werd ingeschakeld in de Authentication-sectie van de appregistratie. Dit is een vereiste voor de OAuth2 device code flow die het exportscript gebruikt. Een *public client* slaat geen clientgeheim (client secret) op: er is geen geheime sleutel die in het script zelf of in een configuratiebestand moet worden bewaard (Microsoft, 2025b). Dit staat in contrast met een *confidential client*, waarbij een geheim wél moet worden bewaard en bij blootstelling (bv. door onbedoeld delen van het script) onmiddellijk een beveiligingsrisico oplevert. De optie *Allow public client flows* werd ingeschakeld in de Authentication-sectie van de appregistratie. Dit is nodig voor de OAuth2 device code flow die het exportscript gebruikt. Een *public client* bewaart geen clientgeheim. Er moet dus geen geheime sleutel in het script of in een configuratiebestand worden opgeslagen (Microsoft, 2025b). Bij een *confidential client* is dat wel nodig, en dat verhoogt het risico wanneer zo'n geheim uitlekt.

De device code flow werkt als volgt: het script vraagt een eenmalige code aan, toont die aan de gebruiker en wacht. De gebruiker opent daarna <https://microsoft.com/devicelogin> in de browser en meldt zich aan. Na die aanmelding krijgt het script een access token. Het wachtwoord zelf komt nooit in het script terecht.

In de praktijk gebeurt dit met `Connect-MgGraph` en de parameter `-UseDeviceCode`, zoals te zien is in codefragment 3.1.

3.3.3. API-permissies: gedelegeerd, niet als applicatie

Deze bachelorproef hanteert één expliciet rechtenmodel voor de exportworkflow.

1) Rechten van de gebruiker De gebruiker krijgt geen algemene tenantbrede leesrechten op OneNote toegewezen. De gebruiker ziet enkel sites waarvan hij lid is. Dit

```

1 Connect-MgGraph -ClientId $ClientId `
2               -TenantId $TenantId `
3               -Scopes "Sites.Read.All","Notes.Read.All" `
4               -UseDeviceCode `
5               -ContextScope Process `
6               -NoWelcome

```

Codefragment 3.1: Device code authenticatie in het exportscript

lidmaatschap wordt vooraf beheerd op de relevante SharedWithCustomer-sites.

2) Rechten van de app De appregistratie OneNote-Bulk-Export krijgt gedelegeerde Graph-scopes (zoals `Notes.Read.All`, `Sites.Read.All` en `User.Read`). In de huidige operationele configuratie is ook `Notes.ReadWrite.All` aanwezig voor compatibiliteit met de bestaande queue-pipeline.

3) Waarom de app deze rechten nodig heeft Microsoft Graph vereist de juiste toegangsrechten om OneNote-inhoud programmatisch te lezen. Zonder die rechten kan het script geen notitieboeken, pagina's en resources ophalen. De app heeft dus functionele Graph-toegang nodig, maar voert acties uit *namens* de aangemelde gebruiker.

4) Effectieve afbakening van toegang De effectieve toegang is de doorsnede van (a) de gedelegeerde app-scopes en (b) de bestaande SharePoint-rechten van de gebruiker. Als de gebruiker geen toegang heeft tot een site, kan het script die site ook niet uitlezen.

Figuur 3.1 toont de gedelegeerde Microsoft Graph-permissies van de gebruikte app-registratie.

API / Permissions name	Type	Description	Admin consent requ...	Status
Microsoft Graph (4)				...
<code>Notes.Read.All</code>	Delegated	Read all OneNote notebooks that user can access	No	Granted for Alistar
<code>Notes.ReadWrite.All</code>	Delegated	Read and write all OneNote notebooks that user can access	No	Granted for Alistar
<code>Sites.Read.All</code>	Delegated	Read items in all site collections	No	Granted for Alistar
<code>User.Read</code>	Delegated	Sign in and read user profile	No	Granted for Alistar

Figuur 3.1: Gedelegeerde Microsoft Graph-permissies die in Microsoft Entra ID aan de appregistratie werden toegekend voor de OneNote-bulk-export.

3.3.4. Admin consent

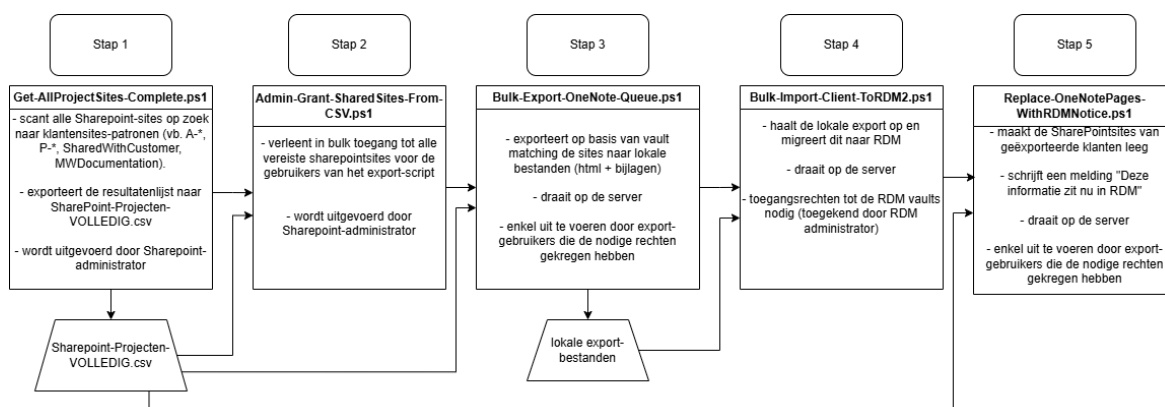
Admin consent wordt eenmalig op tenantniveau verleend voor de appregistratie. Dit keurt het gebruik van de app goed, maar geeft gebruikers niet automatisch toegang tot alle klantsites. Zonder correct SharePoint-lidmaatschap blijft de export geblokkeerd voor die sites.

3.3.5. Operationele scriptvolgorde

De volledige migratie verloopt in vijf operationele stappen:

1. Er wordt eerst een inventaris opgebouwd van relevante klantsites op basis van vaste selectiecriteria.
2. Vervolgens krijgt de exportgebruiker alleen toegang tot de sites die voor de migratie nodig zijn.
3. Daarna wordt de OneNote-inhoud per klant geëxporteerd naar een lokale, werkbare structuur.
4. De exportresultaten worden daarna geïmporteerd in de juiste klantvaults in RDM.
5. Tot slot volgt een nazorgstap waarbij de oude OneNote-omgeving wordt omgeleid naar de nieuwe documentatieplaats in RDM.

Figuur 3.2 toont deze vijf stappen schematisch.



Figuur 3.2: Volledige operationele migratieflow in vijf stappen.

Veiligheidsoverwegingen bij deze aanpak

- De migratie werkt altijd namens een aangemelde gebruiker. Ze kan nooit meer zien of doen dan die gebruiker zelf mag.
- Enkel accounts binnen de eigen tenant kunnen de app gebruiken.
- Sitetoegang wordt per gebruiker en per klant toegekend, alleen voor wat de exporttaak vereist.
- De beheertools loggen wie toegang kreeg en waarvoor, wat traceerbaarheid garandeert.

Deliverable: Een geconfigureerde Entra ID-appregistratie (OneNote-Bulk-Export) met gedelegeerde permissies en admin consent, en de nodige SharePoint-sitetoegang voor de exportgebruiker.

3.4. Fase 4: Ontwikkeling van de migratiescripts

3.4.1. Exportscript: `Export-OneNote-Deduplicated.ps1`

`Export-OneNote-Interactive-Full.ps1` vormde de stabiele basisversie waarop verdere ontwikkeling werd gebouwd. Het script werkt interactief: de gebruiker geeft een zoekterm op (bijvoorbeeld een klantnaam), waarna het script alle overeenkomende SharePoint-sites opzoekt in een CSV-inventaris. Na authenticatie via Microsoft Graph worden de notitieboeken op die sites opgehaald en aan de gebruiker getoond ter bevestiging. Vervolgens voert het script de volledige export uit: pagina-inhoud inclusief tekst en opmaak, afbeeldingen en bijlagen als losse bestanden, checkboxes en de hiërarchie van hoofd- en subpagina's. Throttling-bescherming en een tweefasig herstelproces voor ontbrekende media zorgen ervoor dat de export ook bij grote notitieboeken betrouwbaar verloopt. Het resultaat is een mappenstructuur op schijf per sectie en pagina, klaar voor verdere verwerking.

Tijdens verdere praktijktesten bleek echter dat klantinformatie vaak op meerdere sites dubbel aanwezig is (bijvoorbeeld dezelfde pagina op een projectsite en op een SharedWithCustomer-site). Daarom werd een verbeterde versie ontwikkeld: `Export-OneNote-Deduplicated.ps1`.

Deze versie combineert interactieve siteselectie met intelligente deduplicatie. In eenvoudige termen: het script verzamelt eerst alle pagina's van de relevante sites, groepeert pagina's die inhoudelijk dezelfde titel/sectie vertegenwoordigen, en exporteert daarna alleen de meest recente versie. Zo wordt dubbele of driedubbele informatie niet meer mee gemigreerd.

Een cruciale randvoorwaarde is dat alle gekende SharePoint-sites van een klant effectief worden meegenomen in de discoveryfase. In de praktijk kan een pagina uniek aanwezig zijn op slechts één site; wanneer die site niet gescand wordt, gaat die informatie verloren in de migratie. Door alle sites te scannen en daarna pas te dedupliceren, worden unieke pagina's behouden en wordt verspreide kennis samengebracht op één centrale plaats in RDM.

Het script voert de volgende stappen uit:

1. **Verbinding maken:** authenticatie via device code flow met de geconfigureerde Entra ID-app, gevolgd door een initiële cooldown om throttling te vermijden bij de start van de sessie.
2. **Discovery over meerdere sites:** via een CSV-inventaris worden alle relevante klantensites opgezocht en gescand op notitieboeken, secties en pagina's.
3. **Deduplicatie-index opbouwen:** elke pagina krijgt een sleutel op basis van sectienaam en paginatitel. Pagina's met dezelfde sleutel worden als duplicaten behandeld.

4. **Nieuwste versie selecteren:** bij duplicaten wordt de pagina met de recentste `LastModifiedDate` gekozen voor export.
5. **Hiërarchische export uitvoeren:** de geselecteerde pagina's, media en parent-pagina's worden geëxporteerd met behoud van de structuur.
6. **Throttling-beheer:** het script past adaptief vertragingbeheer toe (Microsoft, 2025a):
 - Minimale vertraging tussen aanvragen
 - Verhoogde vertraging bij HTTP 429-responses
 - Circuit breaker bij opeenvolgende throttle-responses
 - Herstelrondes: maximaal 6 rondes voor mislukte aanvragen, met toenemende wachttijden
7. **Transparantie:** het script schrijft `deduplication-metadata.csv` weg met per pagina de gekozen bron, wijzigingsdatum en duplicate-aantal.

De technische kern van de deduplicatie staat in codefragmenten 3.2, 3.3 en 3.4.

```
1 $allPages += [PSCustomObject]@{
2     SectionName = $section.displayName
3     PageTitle = $page.title
4     LastModifiedDate = $page.lastModifiedDate
5     DeduplicationKey = "$($section.displayName)|$($page.title)"
6 }
```

Codefragment 3.2: Opbouw van een deduplicatiesleutel per OneNote-pagina

```
1 $sorted = $pages | Sort-Object { [DateTime]::Parse($_.
    LastModifiedDate) } -Descending
2 $newest = $sorted[0]
3 $selectedPages += $newest
```

Codefragment 3.3: Selectie van de nieuwste versie bij duplicaten

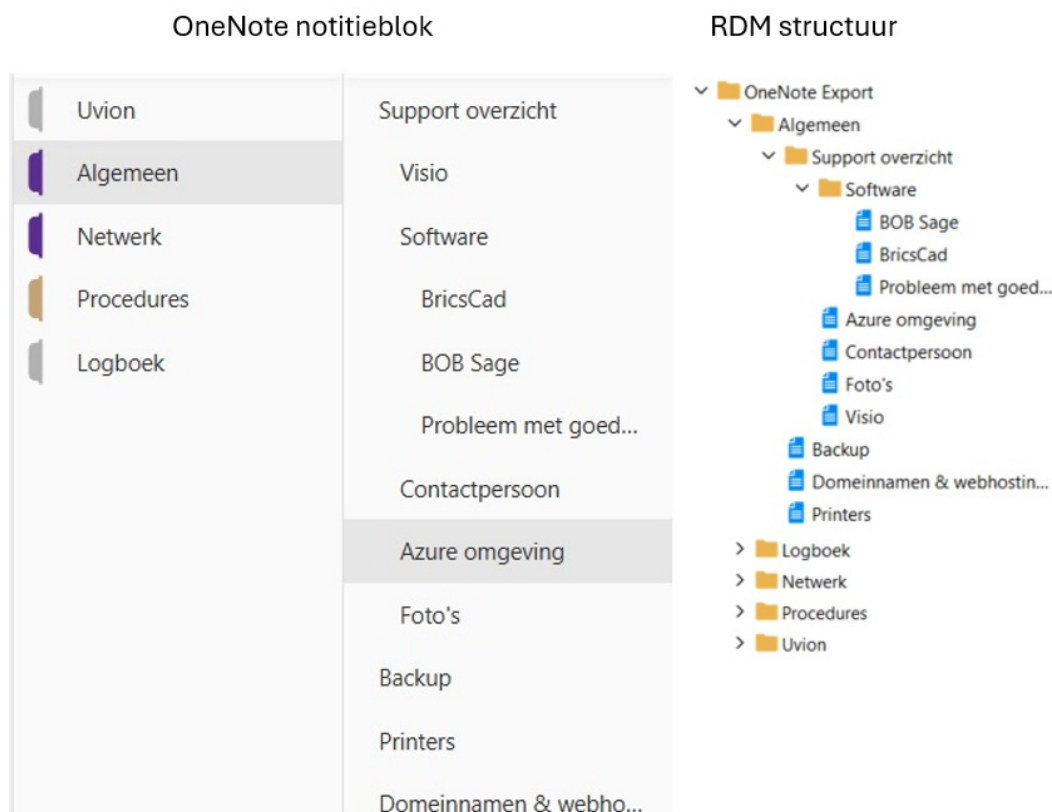
```
1 $metadataPath = Join-Path $exportRoot "deduplication-metadata.csv"
2 $metadata | Export-Csv -Path $metadataPath -NoTypeInfo -Encoding
    UTF8
```

Codefragment 3.4: Wegschrijven van deduplicatie-metadata voor controle en audit

Deliverable: Exportmap per klant met unieke (gededupliceerde) HTML-pagina's en bijbehorende mediabestanden, georganiseerd in de originele hiërarchie van notitieboeken en secties.

3.4.2. Boomstructuur en parent-entiteiten

Naast deduplicatie moest ook de boomstructuur van OneNote intact blijven. Een notebook bestaat niet alleen uit bladeren, maar ook uit tussenliggende pagina's die zelf inhoud kunnen bevatten én tegelijk kinderen hebben. In RDM mogen die pagina's daarom niet verdwijnen tussen folderniveaus: ze moeten als parent-entry behouden blijven, zodat de onderliggende pagina's op dezelfde plaats terechtkomen als in OneNote. Figuur 3.3 toont die vergelijking tussen de bronstructuur links en het resultaat in RDM rechts.



Figuur 3.3: Vergelijking van de hiërarchie in het OneNote-notitieblok en de resulterende structuur in RDM. Parent-pagina's met eigen inhoud worden als entry behouden zodat de nesting identiek blijft aan de bronstructuur.

Het exportscript brengt de volledige paginastructuur in kaart voordat het importsript start. Het importsript gebruikt die informatie om te bepalen hoe elke pagina in RDM moet worden aangemaakt: als gewone documententry of als een entry met onderliggende subpagina's. Dat onderscheid is belangrijk: een pagina die zelf ook subpagina's heeft, moet als parent-entry bewaard blijven. Zonder die parent-entry gaat de inhoud van de pagina verloren of komt de nesting van de structuur niet overeen met het origineel.

Codefragment 3.5 toont hoe het script per pagina bepaalt waar die in RDM terechtkomt en of die pagina zelf nog subpagina's heeft. Die twee gegevens worden sa-

```

1 $relative = $pageDir.FullName.Substring($section.FullName.Length).
  TrimStart('\')
2 $parentPath = if ($relative -like "*\\*") { Split-Path $relative
  -Parent } else { $null }
3
4 $hasChildren = @(
5   Get-ChildItem $pageDir.FullName -Directory -ErrorAction
  SilentlyContinue |
6   Where-Object { Test-Path (Join-Path $_.FullName "page.html") }
7 ).Count -gt 0
8
9 $items.Add([pscustomobject]@{
10   SourcePage      = $entryName
11   TargetFolder    = $targetFolder
12   HasChildren     = $hasChildren
13   HTMLFilePath    = $rdmHtmlPath
14 })

```

Codefragment 3.5: Afleiding van parentpad en kinddetectie voor het behoud van de boomstructuur

men met de paginanaam en het bronbestand bewaard als importmetadata. Het importsript gebruikt die metadata vervolgens om elke pagina op de juiste manier aan te maken in RDM.

```

1 if ($item.HasChildren) {
2   Write-Host "    Page has children -> creating folder with
  attachments" -ForegroundColor Gray
3   $result = Add-AttachmentsToFolder -FolderPath $groupPath
  -FolderName $item.SourcePage -ItemData $item -UpdateIfExists:
  $UpdateHtmlIfExists
4   $resultTag = $result.Tag
5 }

```

Codefragment 3.6: Keuze tussen folder-entry en document-entry op basis van onderliggende pagina's

3.4.3. Analysescript: Analyze-Hydrex-Full.ps1

Tijdens de ontwikkeling werd `Analyze-Hydrex-Full.ps1` voorzien als tussenstap om de exportmap te analyseren en HTML-bestanden te koppelen aan RDM-categorieën via `migration-config.json`. Dat configuratiebestand met categorie-mappings (zoals *Documentatie*, *Credentials*, *Netwerk* en *Servers*) werd handmatig opgebouwd op basis van naamgevingspatronen in de OneNote-notitieboeken. In de effectieve migratieruns van deze case werd deze analysefase echter niet actief ingezet. De categorisatie op basis van dit analysescript behoort dus tot de onder-

zochte aanpak, maar niet tot de finaal gebruikte proof-of-concept flow. AI-gebaseerde categorisatie werd eveneens verkend als mogelijke vervolgstap, maar niet operationeel gebruikt in deze case:

- **Azure OpenAI / Claude AI:** technisch bruikbaar voor inhoudsanalyse, maar minder wenselijk voor deze context omdat vertrouwelijke klantdata naar een externe cloudprovider zou worden verstuurd.
- **Ollama (lokale LLM):** privacyvriendelijk omdat de verwerking lokaal kan gebeuren, maar in de praktijktesten bleek de analysetijd per notitieboek te hoog voor de beoogde migratiesnelheid. Daarom werd ook deze piste niet geactiveerd in de uiteindelijke case-uitvoering.

Voor deze bachelorproef blijven Azure OpenAI en Ollama dus referentieopties voor toekomstig onderzoek, maar buiten scope van de effectief uitgevoerde migratieflow.

3.4.4. Importscript: Bulk-Import-Client-ToRDM.ps1

Het importscript verwerkt de geëxporteerde data en laadt deze in de juiste klant-vault in RDM. De batchvariant Bulk-Import-Client-ToRDM.ps1 bouwt voort op de eerdere importlogica, maar legt meer nadruk op documentatie-integratie, opslagoptimalisatie en robuuste batchverwerking. De werking verloopt als volgt:

1. **Mapkeuze:** via een grafische mappenbrowser (Windows Forms `FolderBrowserDialog`) selecteert de gebruiker de exportmap van de te importeren klant.
2. **Datasource selectie:** het script haalt alle beschikbare RDM-datasources op via de `Devolutions.PowerShell`-module en laat de gebruiker een schrijfbaar datasource kiezen.
3. **Vault selectie:** op basis van de klantnaam (afgeleid uit de mapnaam) worden de beschikbare vaults gescoord op gelijkenis. Bij een score boven 85/100 wordt automatisch de beste match voorgesteld; anders kan de gebruiker handmatig zoeken of kiezen via een lijst.
4. **Importeren:** de geëxporteerde items worden via `Import-DataToRDM` ingevoerd in RDM als *Notes*-entries in de correcte categoriemap van de vault. Bij herhaalde import kan de vlag `-UpdateHtmlIfExists` worden gebruikt om bestaande entries bij te werken.

De automatische vault-voorselectie gebeurt met een eenvoudige scorefunctie op naamgelijkenis, waarna de gebruiker expliciet bevestigt. Dit reduceert het risico op import in de verkeerde vault (codefragment 3.7).

Naast de basisimport werd een extra zoekbaarheidsstap geïntegreerd. In de praktijk bleek dat zoeken in RDM onvoldoende werkte wanneer informatie enkel in de

```
1 $scored = $vaults | ForEach-Object {  
2     [pscustomobject]@{ Vault = $_; Score = Score-Match -Left  
3     $searchTerm -Right $_.Name }  
4 } | Sort-Object Score -Descending  
5 if ($bestScore -ge 85) {  
6     $selectedVault = $bestVault  
7 }
```

Codefragment 3.7: Automatische voorselectie van de RDM-vault op basis van naamscore

HTML-bijlage stond. Daarom wordt de tekstuele inhoud van de OneNote-pagina tijdens de import ook toegevoegd aan de velden *Description*, *DescriptionRtf* en *DocumentationTitle*, zodat gebruikers kunnen zoeken op inhoudstermen en niet alleen op titel.

In `Bulk-Import-Client-ToRDM2.ps1` wordt deze uitgebreide import expliciet geactiveerd via de centrale importaanroep (codefragment 3.8). In de recente runtime wordt `page.rdm.html` bovendien alleen nog als bijlage bewaard wanneer de inline documentatie niet succesvol kan worden ingesteld. Zodra de documentation-editor wél gevuld is, verdwijnt die dubbele kopie, wat grofweg de helft van de opslag voor die pagina scheelt.

```
1 Import-DataToRDM -Data $data `  
2     -DataSourceName $ds.Name `  
3     -VaultName $selectedVault.Name `  
4     -UpdateHtmlIfExists:$UpdateHtmlIfExists
```

Codefragment 3.8: Importaanroep in `Bulk-Import-Client-ToRDM2.ps1`

De zoekindexering zelf gebeurt in de importlogica die door dit script wordt aangeroepen: de geëxtraheerde platte tekst uit HTML wordt in *Description* en *DocumentationTitle* geplaatst, terwijl de leesbare HTML-variant in de documentation-editor terechtkomt (codefragment 3.9).

Daarnaast wordt een leesbare HTML-weergave in de documentation-editor geplaatst, zodat de inhoud niet alleen als bijlage bestaat maar ook direct in de entry zichtbaar en bruikbaar is (codefragment 3.10).

Voor de effectieve zoekwerking moet in RDM ook de veldoptie *Include descriptions in tags* ingeschakeld zijn. Zonder deze instelling wordt de toegevoegde description-tekst niet meegenomen in de tag-gebaseerde zoekindex.

De recente batchruntime voorziet ook een optionele analysetrap waarin `Analyze-Hydrx-Full.ps1` HTML op lokale schijf kan herwerken (zoals beeldcompressie) vóór import. In de effectief uitgevoerde case van deze bachelorproef werd die extra analysetrap niet geactiveerd.

```

1 $description = Build-DocumentationDescription -ItemData $ItemData
2 $Entry.Description = $description
3
4 $searchText = Get-SearchableTextFromHtmlFile -HtmlPath $ItemData.
    HTMLFilePath -MaxChars 2000
5 if (-not [string]::IsNullOrEmpty($searchText)) {
6     $Entry.DocumentationTitle = $searchText.Substring(0, [Math]::Min(
        $searchText.Length, 220))
7 }

```

Codefragment 3.9: Toevoegen van inhoudstekst aan Description, DescriptionRtf en DocumentationTitle

```

1 $docHtml = Build-DocumentationBodyHtml -ItemData $ItemData
2 Set-RDMDocumentationBodyHtml -Entry $existing -HtmlBody $docHtml
    -PageName 'OneNote Import'

```

Codefragment 3.10: Vullen van de documentation-editor met tekstuele OneNote-inhoud

Deliverable: Gevulde klantvault in RDM met alle gemigreerde informatie, georganiseerd per categorie.

3.5. Fase 5: Productiegerichte batchpipeline en ontwerpkeuzes

Na de eerste werkende pipeline werd de oplossing verder uitgebreid naar een productiegerichte batchaanpak met twee kernscripts: `Bulk-Export-OneNote-Queue.ps1` en `Bulk-Import-Client-ToRDM2.ps1`. In deze fase lag de nadruk op schaalbaarheid, fouttolerantie en leesbare operationele flow voor nachtverwerking.

3.5.1. Scriptstructuur en aanroepketen

De batchpipeline is bewust ontworpen volgens een *queue-based execution model*, waarin alle gebruikersinteractie aan het begin van het script verloopt. Dit model stelt beide scripts in staat onbewaakt een lange periode uit te voeren zonder tussenkomst nodig te hebben.

Bij de **exportfase** voert `Bulk-Export-OneNote-Queue.ps1` de volgende stappen uit:

1. De gebruiker selecteert interactief alle OneNote-vaults (SharePoint-sites) waarvan gegevens geëxporteerd moeten worden
2. Deze vaults worden in een interne wachtrij (queue) opgenomen, samen met metagegevens over doellocatie en deduplicatielogica
3. Na deze initiële setup roept het script `Export-OneNote-Deduplicated-`

Queue.ps1 aan, die de vaults aaneengesloten verwerkt

4. Het script loopt onbewaakt door, onafhankelijk van verdere gebruikersinteractie
5. In praktijkdeployment werden deze exports vooral 's nachts ingepland, omdat batchverwerking dan geen verstoringen ervaart van wijzigingen aan de originele OneNote-notitieboeken door actieve gebruikers

Bij de **importfase** hanteert Bulk-Import-Client-ToRDM2.ps1 een identiek patroon:

1. De gebruiker selecteert interactief welke klanten en exportmappen moeten worden geïmporteerd in RDM
2. Het script bouwt een wachtrij van deze klanten op, met vooraf bepaalde RDM-doelvaults en configuratiegegevens
3. De eigenlijke import wordt gedelegeerd aan Import-ToRDM2.ps1, die de vaults in de wachtrij aaneengesloten verwerkt
4. Gedurende de importrun wordt geen verdere gebruikersinteractie vereist; het script voert deduplicatie, groepstructuurcreatie, attachment-embedding en metadatamapping automatisch uit
5. RDM-documentatie, description-velden en losse bijlagen worden in één consistente importstroom samengebracht per klant

Dit queue-based model is cruciaal voor robuuste bulkverwerking: het scheidt gebruikersinteractie (aan het begin) van de eigenlijk langdurige verwerking (server-side, onbewaakt), wat fouttolerantie en logtraceerbaarheid bevordert (Fowler, 2002).

Een aparte analysetrap met Analyze-Hydrax-Full.ps1 werd wel voorzien, maar niet operationeel gebruikt in de finale case-uitvoering.

3.5.2. Operationele logging en crash-diagnostics

Tijdens praktijktesten op de server bleek dat lange batchruns niet altijd stabiel eindigen: in sommige runs stopte het proces onverwacht of sloot het PowerShell-venster abrupt. Zonder persistente logging is dan onduidelijk welke klant al verwerkt was, op welke stap de run faalde, en wat de onderliggende foutmelding was. Daarom werd in zowel de export- als importbatch een doorlopende logstrategie ingebouwd.

In Bulk-Export-OneNote-Queue.ps1 wordt bij scriptstart een unieke logfile met timestamp aangemaakt. Elke statuslijn krijgt een niveau (INFO, WARN, ERROR, SUCCESS) en wordt onmiddellijk naar schijf geschreven, zodat ook bij een onverwachte stop de laatst uitgevoerde stap traceerbaar blijft (codefragment 3.11).

```

1 $script:LogDir = Join-Path $PSScriptRoot "logs"
2 New-Item -ItemType Directory -Path $script:LogDir -Force | Out-Null
3 $script:LogFile = Join-Path $script:LogDir "exportlog$(Get-Date -Format
   'ddMMyyyyHHmmss').log"
4
5 function Write-Log {
6     param(
7         [string]$Message,
8         [ValidateSet('INFO','WARN','ERROR','SUCCESS')][string]$Level =
   'INFO'
9     )
10    $ts = Get-Date -Format 'dd/MM/yyyy HH:mm:ss'
11    $line = "[${ts}] [${Level}] $Message"
12    Add-Content -Path $script:LogFile -Value $line -Encoding UTF8
13 }

```

Codefragment 3.11: Persistente logfile met niveaus in Bulk-Export-OneNote-Queue.ps1

In Bulk-Import-Client-ToRDM2.ps1 werd logging verder uitgebreid met gestructureerde faalregistratie per klant. Bij blijvende fouten na retry worden details niet alleen in de hoofdlog opgenomen, maar ook naar een aparte CSV weggeschreven. Daardoor kan na een crash of gedeeltelijke run gericht worden hervat op de niet-verwerkte of gefaalde items (codefragment 3.12).

```

1 $failedLogPath = Join-Path $failedLogRoot ("{0}-{1}-failed.csv" -f
   $runStamp, $safeCustomer)
2
3 $importStats.FailedItems |
4     Select-Object @{n='Customer';e={$mapping.CustomerName}},
5                 @{n='Vault';e={$vault.Name}},
6                 SourcePage,
7                 TargetFolder,
8                 Category,
9                 Error |
10     Export-Csv -Path $failedLogPath -NoTypeInfo -Encoding UTF8
11
12 Write-Log "[$currentIndex/$totalCount] Gefaalde items voor $($mapping.
   CustomerName):" 'ERROR'
13 foreach ($fi in $importStats.FailedItems) {
14     Write-Log "  MISLUKT: $($fi.SourcePage) [ $($fi.Category)] -> $($fi.
   Error)" 'ERROR'
15 }

```

Codefragment 3.12: Gestructureerde registratie van resterende fouten in Bulk-Import-Client-ToRDM2.ps1

Deze loggingaanpak verhoogt de operationele robuustheid van de batchpipeline: fouten worden achteraf reproduceerbaar geanalyseerd, de voortgang blijft zichtbaar bij onverwachte stops, en reruns kunnen gericht beperkt worden tot probleemitems in plaats van volledige herverwerking.

3.5.3. Fuzzy matching als kern-challenge

In de exportqueue wordt de geselecteerde vaultnaam gebruikt als zoekterm om relevante SharePoint-sites te vinden. Die naam komt niet manueel uit een klant-formulier, maar wordt afgeleid uit de gekozen RDM-vault. Het script zoekt vervolgens naar mogelijke site-matches, toont de beste resultaten en laat de gebruiker de juiste sites bevestigen.

Een terugkerend probleem is dat vaultnamen, foldernamen en SharePoint-sites niet identiek zijn. Daarom gebruikt het exportscript een eenvoudige maar effectieve scorefunctie: eerst normaliseren, daarna exact match, substring-match en ten slotte woordoverlap.

De kern van die scorelogica staat in codefragment 3.13.

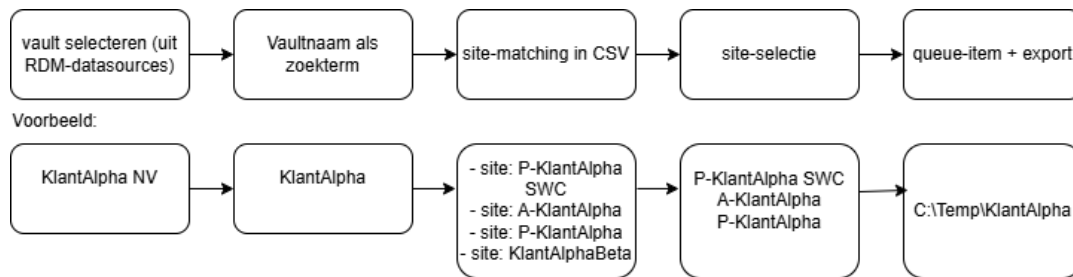
```
1 function Score-Match {
2     param([string]$Left,[string]$Right)
3     $a = Normalize-Name $Left
4     $b = Normalize-Name $Right
5     if ($a -eq $b) { return 100 }
6     if ($a.Length -ge 4 -and $b.Length -ge 4) {
7         if ($a.Contains($b)) { return 95 }
8         if ($b.Contains($a)) { return 90 }
9     }
10    $aw = $a -split '\s+' | Where-Object { $_.Length -gt 2 }
11    $bw = $b -split '\s+' | Where-Object { $_.Length -gt 2 }
12    $common = ($aw | Where-Object { $bw -contains $_ }).Count
13    return [int][math]::Round(($common / [math]::Max($aw.Count,$bw.
14        Count)) * 100, 0)
15 }
```

Codefragment 3.13: Fuzzy matching voor automatische vault-voorselectie (Bulk-Export-OneNote-Queue.ps1)

Deze aanpak bleek in de praktijk snel en begrijpelijk voor operators. Bij twijfel blijft manuele bevestiging mogelijk, waardoor het risico op import in de verkeerde vault beperkt blijft. Een geanonimiseerde voorbeeldoutput van deze matchselectie staat in figuur 3.4.

3.5.4. Onbewaakte batchverwerking en betrouwbaarheid

Voor nachtverwerking moet een script volledig onbewaakt kunnen draaien. De Bulk-Export-OneNote-Queue.ps1 implementeert hiervoor een gestructureerde



Figuur 3.4: Geanonimiseerde voorbeeldoutput van vaultnaam-naar-site-matching in de export-queue.

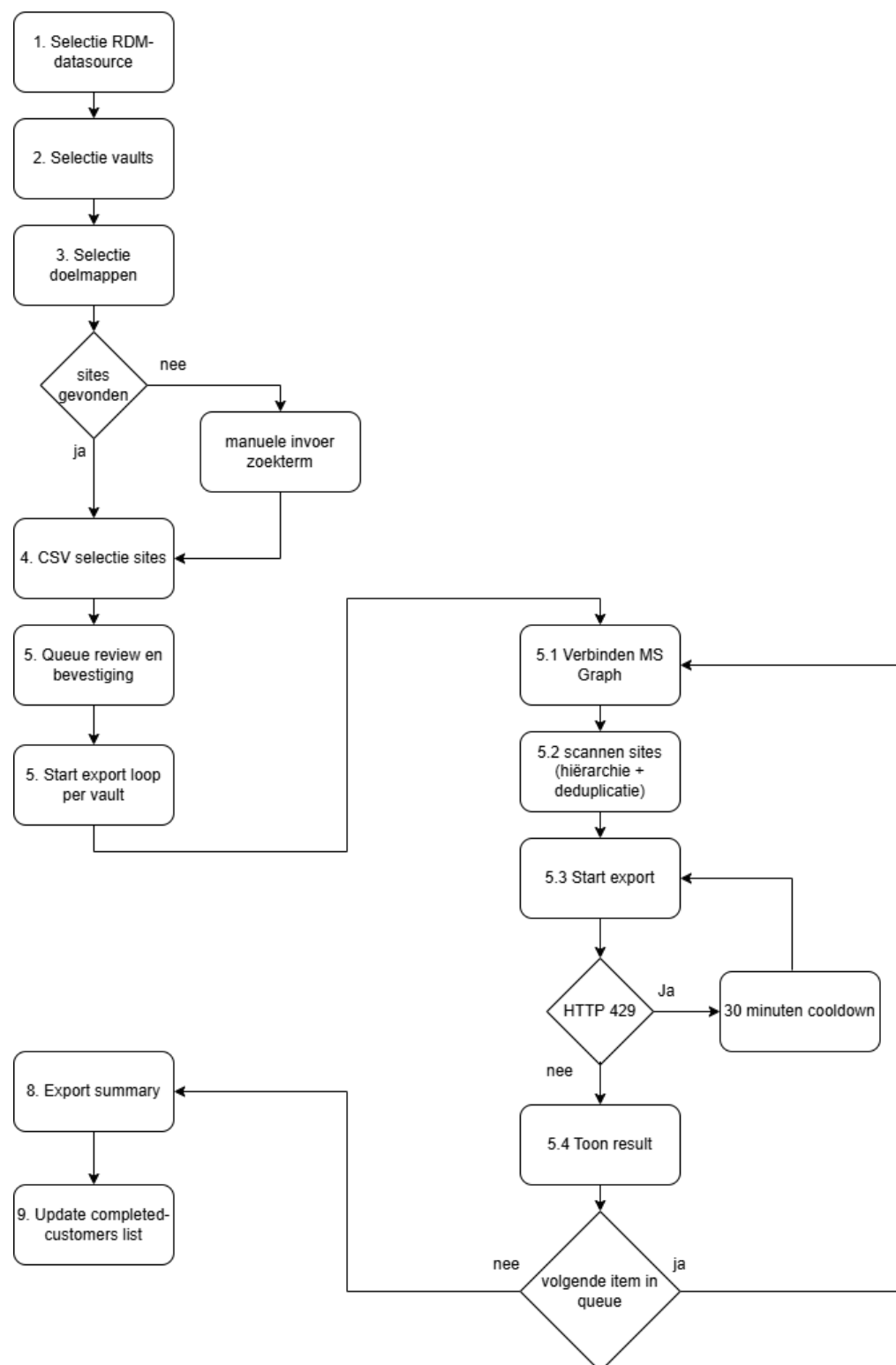
flow, weergegeven in figuur 3.5.

1. **Interactieve voorbereiding** (stappen 1–3): gebruiker selecteert RDM-datasource, vaults en doelmappen.
2. **Sitebepaling met fallback**: na de initiële sitezoeking volgt een beslismoment (“sites gevonden”); bij geen match wordt eerst een manuele zoekterm ingevoerd.
3. **Definitieve siteselectie** (stap 4): de gevonden sites worden bevestigd via de CSV-selectie.
4. **Queue review** (stap 5): gebruiker bevestigt de volledige exportconfiguratie vóór de verwerking start.
5. **Exportloop per vault** (stappen 5.1–5.4): voor elk item in de queue automatisch:
 - Verbinding maken met Microsoft Graph API
 - Sites scannen op hiërarchie en detecteren duplicaten
 - Export starten via `Export-OneNote-Deduplicated-Queue.ps1`
 - Bij HTTP 429 (throttling): 30 minuten cooldown, daarna automatische herpoging
 - Resultaat (succes/faal) registreren en volgende item verwerken
6. **Afronding** (stappen 8–9): samenvattingsrapport tonen en de lijst met succesvol verwerkte klanten bijwerken.

Een gekend Windows-probleem is dat ConHost pauzeert bij een muisklik op het venster (Quick Edit Mode). Het script schakelt dit expliciet uit aan de start:

Door deze kleine ingreep stopt de batch niet meer onverwacht door gebruikersinteractie. In combinatie met automatische retry-stappen en throttle-afhandeling verhoogt dit de operationele betrouwbaarheid aanzienlijk.

De import-run volgt een ander patroon dan de export: het script voert alle setup- en selectiefasen interactief uit aan het begin. Daarna worden alle geselecteerde



Figuur 3.5: Flowchart van de procespijplijn in Bulk-Export-OneNote-Queue.ps1: van datasource-selectie tot export per vault en automatische 429-retry met 30-minuten cooldown.

```

1 try {
2     $consoleHelperCode = @"
3 using System;
4 using System.Runtime.InteropServices;
5 public class ConsoleQuickEdit {
6     const uint ENABLE_QUICK_EDIT = 0x0040;
7     const uint ENABLE_EXTENDED_FLAGS = 0x0080;
8     // ... GetConsoleMode / SetConsoleMode ...
9 }
10 "
11 if (-not ([System.Management.Automation.PSTypeName]'
    ConsoleQuickEdit').Type) {
12     Add-Type -TypeDefinition $consoleHelperCode
13 }
14 [ConsoleQuickEdit]::Disable()
15 } catch { }
```

Codefragment 3.14: Quick Edit uitschakelen om onbewaakte runs robuust te maken

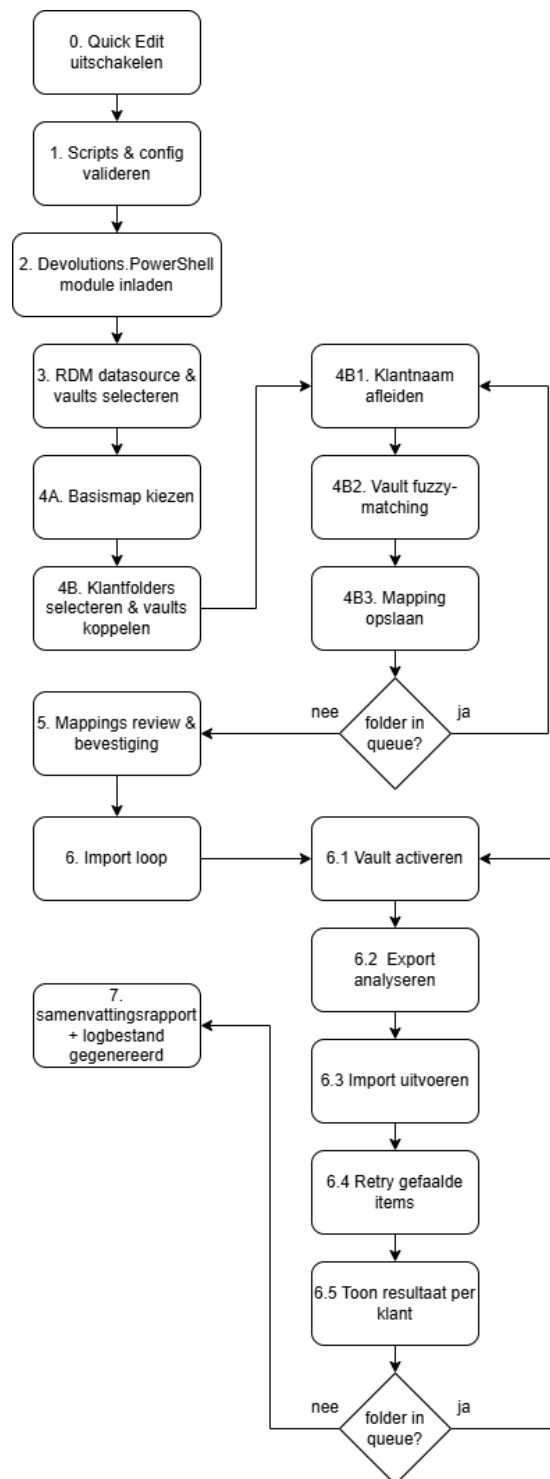
klantfolders volledig automatisch verwerkt. De kern van de import-fase bestaat uit vier stappen per klant (zie figuur 3.6):

1. **Vault activeren:** De geselecteerde RDM-vault activeren zodat volgende imports daarin belanden.
2. **Exportdata voorbereiden:** De gededuplicateerde exportmap inlezen en valideren als bron voor import.
3. **Import uitvoeren:** Per geëxporteerde pagina één HTML-document in de vault importeren, met optionele duplicate-detectie en intelligente foutafhandeling.
4. **Retry gefaalde items:** Wanneer items falen, kunnen ze opnieuw geprobeerd worden in parent-first volgorde (om folder-hiërarchie-problemen te voorkomen).

Na afronding genereert het script een gedetailleerd samenvattingsrapport met logbestanden en een CSV per klant met resterende fouten (indien aanwezig).

3.5.5. Praktische serveropzet voor langdurige batchruns

Voor de operationele uitvoering over langere periodes werd door Internal IT een aparte server voorzien. Op die server kregen een beperkt aantal gebruikers de nodige lees- en schrijfrechten, en administratorrechten voor het uitvoeren van de migratiescripts. In de praktijk bleek die administratorcontext noodzakelijk om de scripts stabiel en volledig te laten doorlopen in deze omgeving.



Figuur 3.6: Flowchart van de batch import-pijplijn in Bulk-Import-Client-ToRDM2.ps1: van setup en config-validatie tot multi-select klantfolder-mapping en per-vault import met optionele retry.

De keuze voor een vaste server is bewust gemaakt. Uitvoering vanaf een persoonlijke laptop is minder geschikt omdat die niet continu op het kantornetwerk zit en thuisgebruik afhankelijk is van een actieve VPN-verbinding. Voor nachtverwerking en langere runs verhoogt dat het risico op onderbrekingen door netwerk- of sessieproblemen.

De werkmethode verloopt in opeenvolgende golven:

1. **Dag 1 (nacht):** een exportbatch voor ongeveer 50 klanten via `Bulk-Export-OneNote-Queue.ps1`.
2. **Dag 2:** import van diezelfde klantgroep naar RDM via `Bulk-Import-Client-ToRDM2.ps1`.
3. **Na import:** uitvoeren van `Replace-OneNotePages-WithRDMNotice.ps1` voor dezelfde klanten.

Tabel 3.1: Praktische dagplanning voor servergebaseerde migratiegolven.

Moment	Fase	Script	Doel
Dag 1 (nacht)	Export	<code>Bulk-Export-OneNote-Queue.ps1</code>	Ongeveer 50 klanten exporteren naar lokale bestanden
Dag 2 (overdag)	Import	<code>Bulk-Import-Client-ToRDM2.ps1</code>	Dezelfde klantgroep in RDM importeren
Direct na import	Nazorg	<code>Replace-OneNotePages-WithRDMNotice.ps1</code>	Oude OneNote-pagina's overschrijven met RDM-verwijzing

Het is belangrijk dat deze stappen kort op elkaar volgen. Hoe langer de periode tussen export, import en replace, hoe groter de kans dat gebruikers intussen nog nieuwe informatie in OneNote toevoegen. Daarom is de instructie aan betrokken gebruikers eenvoudig: vanaf de geplande migratiebatch geen nieuwe informatie meer in de betrokken OneNote-notitieboeken registreren en na de import uitsluitend in RDM werken.

3.5.6. Data integrity en duplicate detection

Het exportscript voert duplicate-detectie uit voordat de effectieve export start. In `Export-OneNote-Deduplicated-Queue.ps1` worden alle pagina's eerst samengebracht in een deduplicatie-index op basis van `SectionName` en `PageTitle`. Wan-

neer meerdere sites dezelfde pagina bevatten, wordt alleen de recentste versie geëxporteerd. Zo blijft de export compact en worden dubbele pagina's al aan de bron weggefilterd.

```
1 $deduplicationIndex = Build-DeduplicationIndex -AllPages
   $allPagesFromAllSites
2 $selectionResult = Select-NewestPages -DeduplicationIndex
   $deduplicationIndex
3
4 $selectedPages = $selectionResult.Pages
5 $metadata = $selectionResult.Metadata
6
7 $metadataPath = Join-Path $ExportPath "deduplication-metadata.csv"
8 $metadata | Export-Csv -Path $metadataPath -NoTypeInfoInformation -Encoding
   UTF8
```

Codefragment 3.15: Duplicate-detectie en selectie van de nieuwste versie in de exportqueue

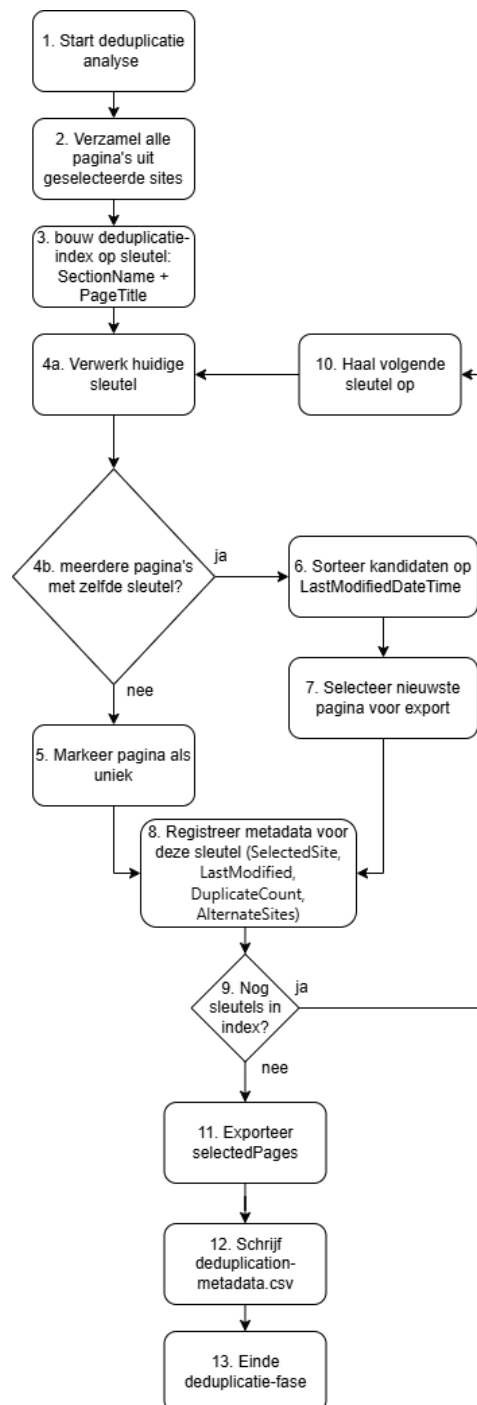
In codefragment 3.15 wordt de deduplicatie in vier duidelijke stappen uitgevoerd. Eerst bouwt `$deduplicationIndex = Build-DeduplicationIndex -AllPages $allPagesFromAllSites` een index op basis van alle gevonden pagina's; pagina's met dezelfde sleutel (sectie + titel) komen in dezelfde groep terecht. Daarna kiest `Select-NewestPages` per groep precies één winnaar, namelijk de recentste versie. Het resultaat wordt opgesplitst in `$selectedPages` (de pagina's die effectief geëxporteerd worden) en `$metadata` (auditinformatie over welke alternatieven afvielen). Met `$metadataPath = Join-Path $ExportPath "deduplication-metadata.csv"` bepaalt het script waar het rapportbestand komt te staan. De laatste regel `Export-Csv` schrijft die metadata als CSV weg, zodat de export alleen de meest recente versie per pagina bevat en achteraf transparant verifieerbaar blijft welke site werd geselecteerd en hoeveel alternatieven zijn weggefallen.

Figuur 3.7 illustreert deze stappen visueel: van het groeperen van mogelijke duplicaten tot de selectie van precies één recentste versie per paginaklasse en de registratie in metadata.

3.5.7. API rate limiting en intelligent backoff

Bij Microsoft Graph-verkeer traden regelmatig 429-fouten op. In plaats van direct te falen, bouwt de queue een gecontroleerde cooldown in en probeert daarna opnieuw.

Codefragment 3.16 vangt fouten centraal op in een catch-blok. De variabele `$errorMessage` bevat de fouttekst van de exception. Met `$isThrottle` controleert het script via een regex-patroon (`429|Too Many Requests|throttle`) of het om rate limiting gaat. Als dat zo is en het aantal pogingen (`$attempt`) nog onder de limiet (`$maxAttempts`) zit, start `Start-QueueThrottleCooldown -Seconds 1800` een



Figuur 3.7: Flowchart en voorbeeldmatrix van duplicate-detectie op exportniveau: per Section-Name/PageTitle-combinatie wordt alleen de recentste versie geselecteerd voor export.

```
1 } catch {
2     $errorMessage = $_.Exception.Message
3     $isThrottle = $errorMessage -match '429|Too Many Requests|throttle'
4
5     if ($isThrottle -and $attempt -lt $maxAttempts) {
6         Start-QueueThrottleCooldown -Seconds 1800 `
7             -Reason "429 tijdens queue item '$($item.VaultName)'"
8         continue
9     }
10    throw
11 }
```

Codefragment 3.16: Throttle-afhandeling met cooldown in de export-queue

wachttijd van 30 minuten. Het sleutelwoord `continue` zorgt er daarna voor dat dezelfde klant opnieuw geprobeerd wordt in plaats van de volledige run te stoppen. Alleen wanneer de fout geen throttle-geval is, of wanneer alle pogingen opgebruikt zijn, voert `throw` de fout opnieuw uit zodat die zichtbaar blijft in de rapportering. Praktisch betekende dit dat lange runs vaker volledig klaar waren tegen de ochtend, ook wanneer de tenant tijdelijk streng throttelde.

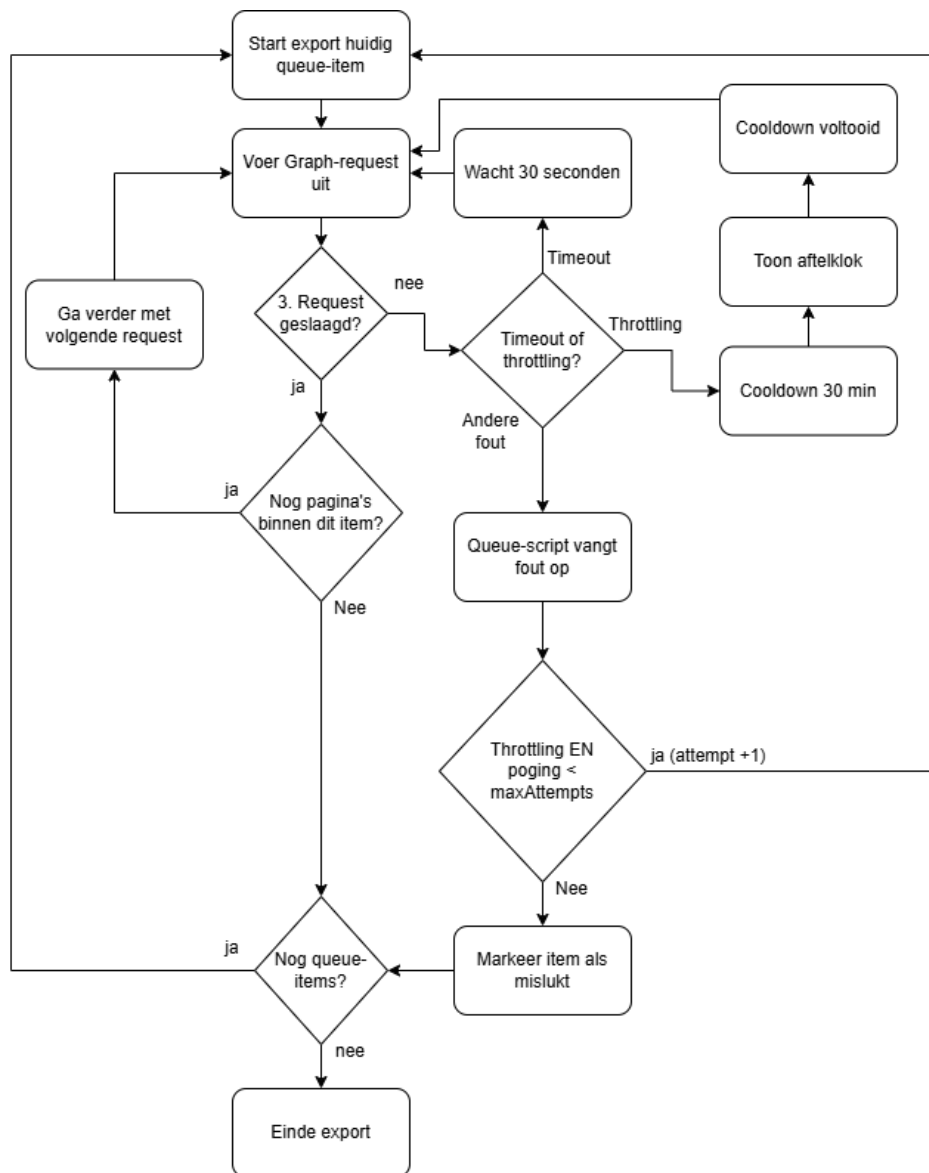
3.5.8. Configuration-driven architecture

De batchimport valideert vooraf of alle afhankelijkheden en configuratie aanwezig zijn. Hierdoor faalt het script vroeg en duidelijk, in plaats van halverwege de run.

```
1 $ImportScript = Join-Path $PSScriptRoot "Import-ToRDM2.ps1"
2 $AnalyzeScript = Join-Path $PSScriptRoot "Analyze-Hydrex-Full.ps1"
3 $ConfigPath = Join-Path $PSScriptRoot "migration-config.json"
4
5 if (-not (Test-Path $ImportScript)) { throw "Import-ToRDM2.ps1 niet
6     gevonden." }
7 if (-not (Test-Path $AnalyzeScript)) { throw "Analyze-Hydrex-Full.ps1
8     niet gevonden." }
9 if (-not (Test-Path $ConfigPath)) { throw "migration-config.json
10     niet gevonden." }
11
12 $script:UserConfig = Get-Content $ConfigPath -Raw -Encoding UTF8 |
13     ConvertFrom-Json
```

Codefragment 3.17: Config-gedreven startup met harde validatie van scripts en JSON-config

In codefragment 3.17 worden eerst drie paden opgebouwd vanuit `$PSScriptRoot`: `$ImportScript`, `$AnalyzeScript` en `$ConfigPath`. Dat zijn dus respectievelijk het importscript, het analysescript en het JSON-configbestand die in dezelfde scriptmap verwacht worden. Met `Test-Path` controleert het script vervolgens



Figuur 3.8: Cooldown en voortgang bij HTTP 429 of andere throttling.

of elk bestand effectief bestaat. Faalt één controle, dan stopt de run meteen met `throw` en een concrete foutmelding. Pas wanneer alle vereisten aanwezig zijn, wordt de inhoud van `migration-config.json` ingelezen met `Get-Content -Raw` en omgezet naar een bruikbaar object via `ConvertFrom-Json`, opgeslagen in `$script:UserConfig`.

Dit maakt de pipeline voorspelbaar: de run start alleen wanneer de context correct is. De volledige opstartcontrole is visueel weergegeven in figuur 3.9.

3.5.9. Two-phase architecture (export + import)

De architectuur is bewust opgesplitst in twee fases. Eerst wordt data per klant geëxporteerd naar bestanden. Daarna wordt die output geïmporteerd in RDM. Deze scheiding maakt troubleshooting eenvoudiger en laat hergebruik van exportbestanden toe.

Belangrijk is dat fase 1 volledig afgerond wordt voordat fase 2 start. Daardoor liggen alle outputbestanden, klantmappen, logs en metadata stabiel op schijf en hoeft de importfase niets meer op te halen uit OneNote of Graph. Praktisch betekent dit dat een fout in de importfase geen nieuwe export vereist: de operator kan fase 2 herstarten op dezelfde bestanden, wat troubleshooting versnelt en het risico op inconsistentie tussen runs verlaagt.

In de praktijk worden beide fases meestal sequentieel uitgevoerd: eerst de volledige bulk-export, daarna de bulk-import op de gegenereerde exportbestanden. Dit verlaagt operationeel risico, maakt tussentijdse kwaliteitscontrole mogelijk en laat herstart van fase 2 toe zonder fase 1 opnieuw uit te voeren.

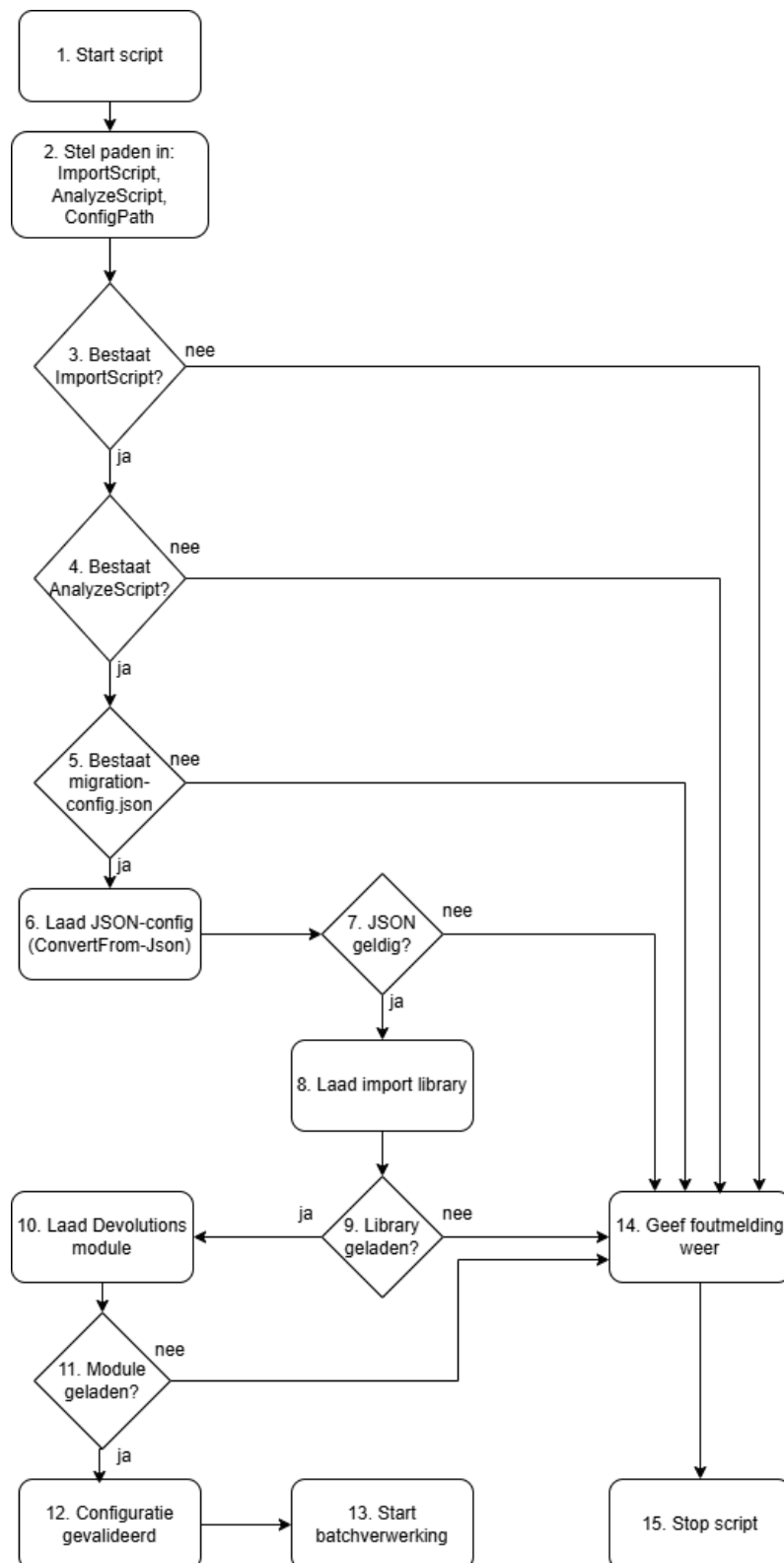
3.5.10. HTML reconstruction en moeilijkheden met image magic bytes

Een belangrijk praktijkprobleem zat in media-bestanden zonder bruikbare extensie of met MIME `application/octet-stream`. Zonder extra detectie kwamen deze bestanden in output terecht als `.bin` en werden ze later in RDM als onbekend bestandstype behandeld.

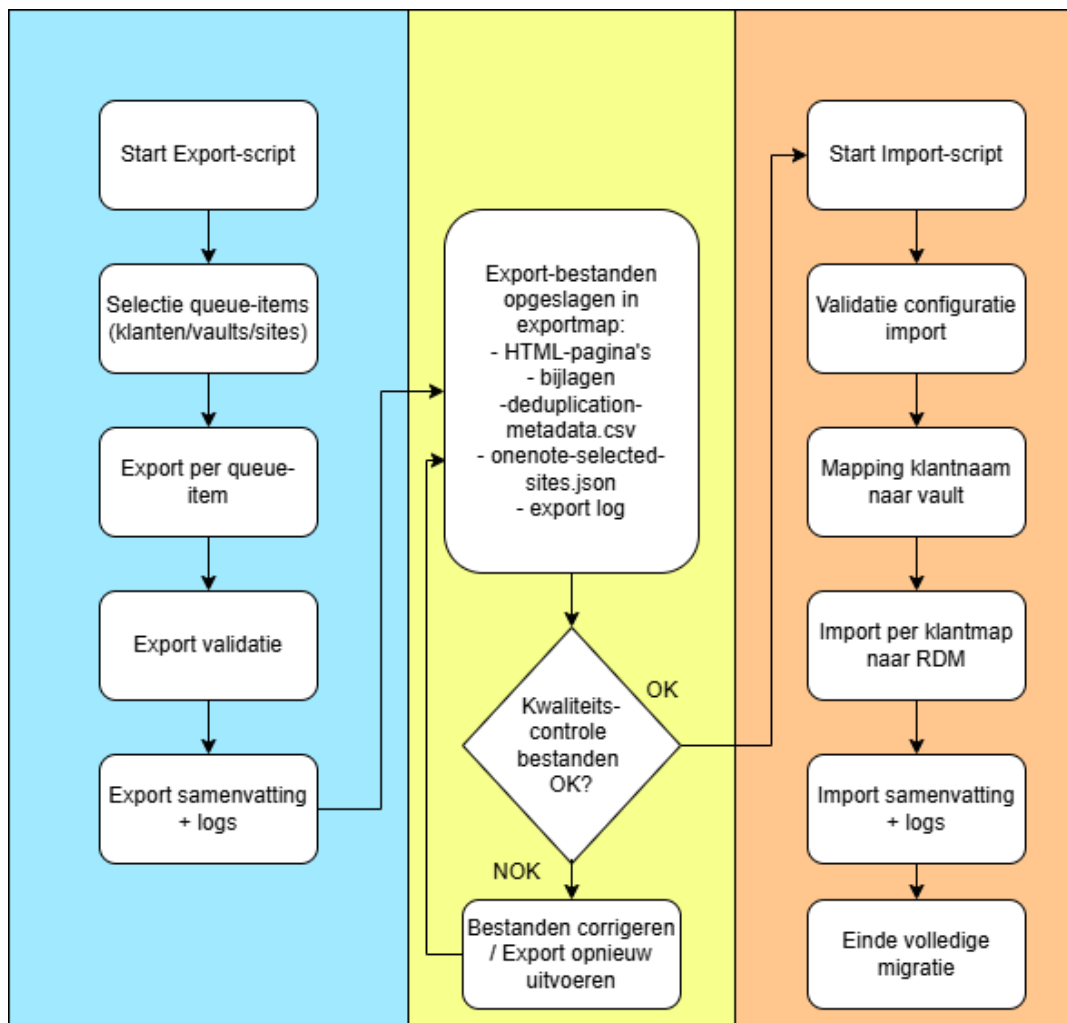
De MIME-informatie (Multipurpose Internet Mail Extensions – een standaard voor bestandstypes) komt uit de Graph API en beschrijft het type van een bestand. In deze migratie werd deze informatie gebruikt als eerste poging om een geschikte extensie toe te kennen. Echter, die informatie is niet altijd betrouwbaar: veel bestanden krijgen het generieke type `application/octet-stream` (ongespecificeerd binair), of de MIME-hint ontbreekt geheel. Daarom was een fallback naar magic bytes (de binaire signatuur van het bestandsbegin) nodig.

De oplossing bestond uit twee stappen:

1. eerst proberen een extensie af te leiden uit MIME;
2. bij mislukking of bij `.bin` de eerste bytes lezen (magic bytes) en het bestand hernoemen naar de juiste extensie.



Figuur 3.9: Validatie van scripts en configuratie vóór de start van de import.



Figuur 3.10: Twee-fasen architectuur van de migratie: eerst worden alle OneNote-gegevens per klant geëxporteerd naar bestanden, daarna worden die exportbestanden in een aparte fase geïmporteerd in RDM.

```

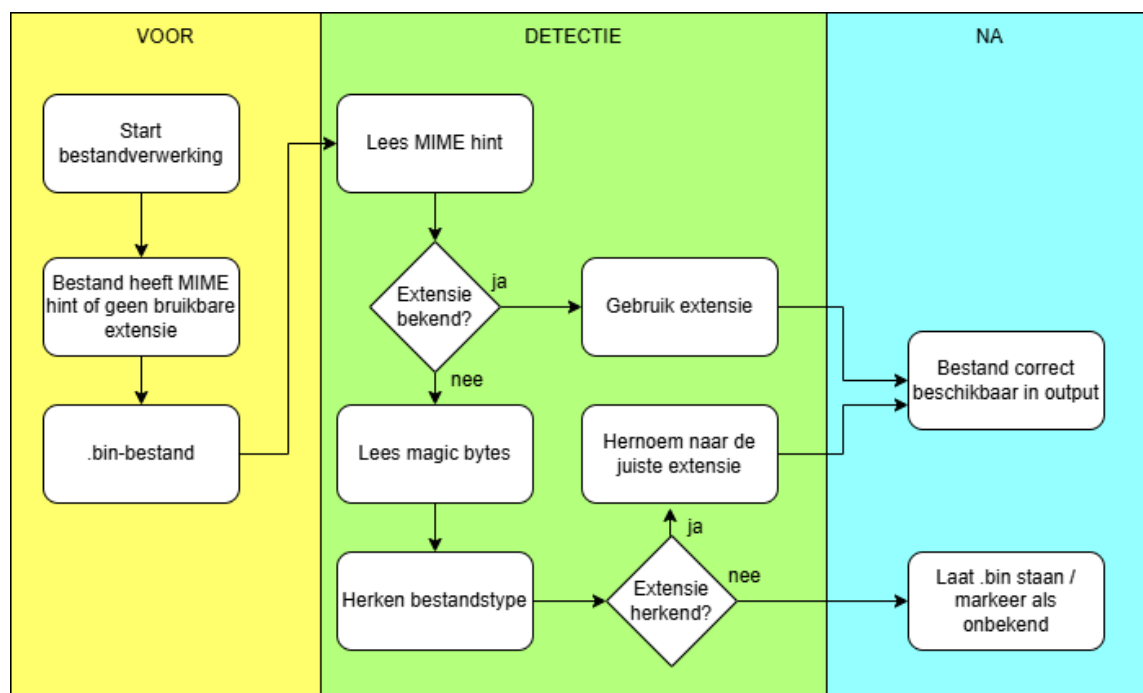
1 $hintExt = MimeToExt $Resource.MimeHint
2 if (-not $hintExt) { $hintExt = ".bin" }
3
4 if ($dl.Success -and $target.Ext -eq ".bin") {
5     $dlBytes = [System.IO.File]::ReadAllBytes($target.FullPath)
6     $detectedExt = DetectExtFromMagicBytes $dlBytes
7     if ($detectedExt -and $detectedExt -ne ".bin") {
8         $newFullPath = [System.IO.Path]::ChangeExtension($target.
9             FullPath, $detectedExt)
10         Rename-Item -LiteralPath $target.FullPath -NewName (Split-Path
11             $newFullPath -Leaf)
12     }
13 }
  
```

Codefragment 3.18: Van onbekend .bin-bestand naar correct type via magic bytes

Codefragment 3.18 werkt in twee lagen. Eerst probeert `$hintExt = MimeToExt $Resource.MimeHint` een extensie af te leiden uit de MIME-hint. Als die ontbreekt of niet bruikbaar is, krijgt `$hintExt` tijdelijk de waarde `.bin`. Daarna volgt de fallback: alleen als de download geslaagd is (`$dl.Success`) en het doelbestand nog `.bin` is (`$target.Ext -eq ".bin"`), leest het script de eerste bytes van het bestand in `$dlBytes`. De functie `DetectExtFromMagicBytes` vergelijkt die bytes met gekende bestandsheaders en geeft een vermoedelijke extensie terug in `$detectedExt`. Wanneer die extensie geldig is en niet opnieuw `.bin`, bouwt `$newFullPath` het nieuwe pad op en hernoemt `Rename-Item` het bestand naar de correcte extensie.

Concreet herkent `DetectExtFromMagicBytes` onder andere JPEG, PNG, GIF, PDF, ZIP-gebaseerde Office-formaten en OLE-formaten. Dit was cruciaal om afbeeldingen correct op te slaan en opnieuw weer te geven in de doelomgeving.

Figuur 3.11 illustreert dit effect met een voor/na-vergelijking van detectie en bestandsherkenning.



Figuur 3.11: Voor en na de detectie van het juiste bestandstype via magic bytes.

3.5.11. Phased rollout en change management

Voor uitrol op veel klanten is gefaseerd werken noodzakelijk. Het exportscript houdt daarom automatisch bij welke klanten al succesvol verwerkt zijn.

De kern van deze opvolglogica staat in codefragment 3.19.

Dit ondersteunt een veilige rolloutstrategie: kleine batches starten, fouten analyseren, en daarna opschalen naar grotere golven. Succesvol geëxporteerde klanten worden opgeslagen in `bulk-export-completed-customers.txt`. Bij volgende

```

1 [string]$CompletedCustomersFile = ".\logs\
   bulk-export-completed-customers.txt"
2
3 $successfulNames = @($ExecutionContext.Results |
4   Where-Object { $_.Status -eq 'Success' } |
5   ForEach-Object { [string]$_VaultName } |
6   Select-Object -Unique)
7
8 $successfulNames | Add-Content -Path $resolvedPath

```

Codefragment 3.19: Persistente opvolging van succesvol afgewerkte klanten voor gefaseerde uitrol

runs leest het script dit bestand en voegt alleen nieuwe klanten toe, waardoor gefaseerde migratie over meerdere weken mogelijk is.

3.5.12. Nazorg: OneNote-pagina's vervangen door RDM-verwijzing

Na export en import blijft er een belangrijk adoptierisico bestaan: gebruikers kunnen de oorspronkelijke OneNote-notitieboeken blijven gebruiken, waardoor informatie opnieuw over twee systemen verspreid raakt. Om die situatie te vermijden werd het script `Replace-OneNotePages-WithRDMNotice.ps1` toegevoegd als nazorgstap.

Het script overschrijft bestaande OneNote-pagina's met een korte verwijzing naar RDM (standaard: *"Deze informatie zit nu in RDM."*). Zo wordt voor eindgebruikers ondubbelzinnig duidelijk dat nieuwe of gewijzigde informatie voortaan in RDM moet worden geregistreerd. De uitvoering gebeurt per geselecteerd notitieboek en per pagina, en vereist expliciete gebruikersbevestiging.

Technisch overschrijft de helperfunctie de originele pagina-inhoud en herpubliceert ze met dezelfde titel en de RDM-melding. Deze stap vereist daarom schrijfpermissies op OneNote (`Notes.ReadWrite` of `Notes.ReadWrite.All`) en wordt pas na validatie van de migratie toegepast.

Deliverable: Gecontroleerde nazorgprocedure met `Replace-OneNotePages-WithRDMNotice.ps1`, waarmee gemigreerde OneNote-pagina's na validatie worden vervangen door een duidelijke RDM-verwijzing.

3.6. Fase 6: Validatie en aanbevelingen

De proof-of-concept werd getest door de volledige migratiepipeline uit te voeren voor een selectie van klantnotitieboeken. Daarbij werd niet alleen gekeken of de scripts technisch werkten, maar ook of het resultaat bruikbaar bleef in de dagelijkse praktijk. De validatie omvatte:

- Controle van de structuurintegriteit: secties en subpagina-hiërarchie correct aanwezig in RDM?

- Controle van mediabestanden: zijn alle afbeeldingen en bijlagen correct opgehaald en zichtbaar in de HTML-inhoud in RDM?
- Operationele opvolging van het migratietempo via onbewaakte runs buiten de werkuren, zodat verwerking kon doorgaan zonder impact op de dagelijkse werking van medewerkers

Op basis van de bevindingen werden enkele aanbevelingen geformuleerd voor verdere uitrol. Daarin kwamen vooral twee punten terug: de toegangsrechten nog gericht beperken en nagaan of Ollama bruikbaar is voor privacyveilige AI-categorisatie. Dat zijn geen bijkomstige details, maar concrete verbeterpunten voor een volgende versie van de oplossing.

De validatiefase toonde aan dat de migratie niet alleen technisch moest werken, maar ook bruikbaar moest blijven voor de eindgebruiker. Het volstond dus niet dat bestanden werden overgezet; de resulterende structuur moest ook logisch blijven en makkelijk onderhoudbaar zijn. Vooral de combinatie van hiërarchie en mediabestanden bleek daarbij belangrijk. Als die twee elementen niet goed samenvallen, voelt RDM voor de gebruiker al snel aan als een losse opslagplaats in plaats van als een echte vervanging.

Verder werd duidelijk dat de grootste beperkingen niet in de basislogica lagen, maar in praktische randvoorwaarden zoals toegangsbeheer en verwerkingstijd. Voor een latere uitrol zijn dus vooral verbeteringen nodig op het vlak van rechtenbeheer, batchplanning en de afhandeling van uitzonderingen. Dat betekent bijvoorbeeld dat toegang nog gericht moet worden toegekend, terwijl de planning van export- en importgolven in de praktijk al goed afgestemd is en de voornaamste beperking vooral ligt bij de exportverwerkingstijd door throttling van Microsoft Graph. De proof-of-concept vormt daarmee een werkbare basis, maar toont ook waar de grenzen van de huidige aanpak liggen.

Ten slotte blijft een beperkte manuele controle nuttig, zeker bij de eerste batches. Zelfs met deduplicatie, magic-byte-detectie en gefaseerde uitrol is het verstandig om nog na te gaan of de resultaten overeenstemmen met de verwachting. Die controle hoeft niet zwaar te zijn, maar helpt wel om fouten vroeg te zien en bij te sturen waar nodig. Zo blijft automatisering een hulpmiddel voor kwaliteit, en geen vervanging van kritische opvolging.

4

Conclusie

Deze bachelorproef laat zien dat een geautomatiseerde migratie van OneNote naar RDM binnen E.S.C. BV technisch haalbaar is, als de aanmelding goed is opgezet, throttling correct wordt afgehandeld en de importflow duidelijk blijft. De ontwikkelde pipeline zet verspreide OneNote-inhoud om naar een centralere en beter beheersbare RDM-structuur, in lijn met PAM-principes zoals least privilege, logging en gecontroleerde toegang.

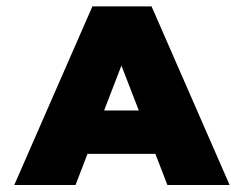
Een belangrijke meerwaarde zit in de deduplicatiestap. Door dubbele en driedubbele pagina's vooraf te detecteren en alleen de meest recente versie te exporteren, wordt minder data verwerkt en is de kans kleiner dat er dubbele of tegenstrijdige informatie in RDM belandt. Tegelijk blijkt wel dat die aanpak alleen goed werkt als de discoveryfase alle relevante SharePoint-sites per klant meeneemt. Een pagina kan immers op één site bestaan en op een andere site ontbreken. Pas wanneer eerst alles volledig wordt gescand en daarna pas wordt gededupliceerd, ontstaat een betrouwbaar resultaat.

Ook de structuur van de bron speelt een grote rol. OneNote is historisch gegroeid in een flexibele maar losse vorm, terwijl RDM meer expliciete ordening vraagt. Dat is tegelijk een voordeel en een uitdaging: de vaste structuur maakt documentatie beter beheerbaar, maar vraagt wel dat bestaande informatie zorgvuldig wordt overgezet. De waarde van de ontwikkelde pipeline zit dus vooral in het bewaren van die structuur. Ze verplaatst niet gewoon bestanden, maar zet losse OneNote-informatie om in een overzichtelijke eindstructuur in RDM.

Daaruit volgt ook dat automatisering in zo'n migratie vooral nuttig is wanneer ze herhalende taken overneemt die anders veel tijd kosten of foutgevoelig zijn. Deduplicatie, boomstructuurbehoud, bestandstypeherkenning en gefaseerde uitrol zijn daar goede voorbeelden van. Geen van die onderdelen is op zichzelf spectaculair, maar samen maken ze het verschil tussen een losse test en een workflow die echt bruikbaar is in productie. De meerwaarde van dit werk zit dus minder in één apart

script dan in de combinatie van keuzes die samen een stabiele aanpak vormen. Voor verder werk zijn er nog duidelijke verbeterpunten zoals: een nauwkeurigere afbakening van permissies, meer automatisering van categorisatie zonder privacy-verlies, en betere opvolging van lange migratiebatches. Als die punten verder worden uitgewerkt, kan de huidige proof-of-concept nog robuuster worden. Binnen de scope van deze bachelorproef is alvast aangetoond dat de basisaanpak klopt en dat de migratie technisch en organisatorisch verdedigbaar is.

Tot slot is het resultaat bruikbaar als vertrekpunt voor gelijkaardige migraties binnen de organisatie. De keuzes rond structuurbehoud, deduplicatie, permissiemodel en nazorg vormen een duidelijke basis voor toekomstige documentatiemigraties bij E.S.C. BV. De kern blijft daarbij eenvoudig: een migratie is pas echt geslaagd als de informatie niet alleen verplaatst wordt, maar ook vlot bruikbaar blijft in het doelplatform. Zo zijn er andere teams binnen hetzelfde bedrijf die momenteel ook gebruik maken van de OneNotes. Hun secties zijn blijven bestaan na de export, zij zouden de overstap echter ook kunnen maken naar RDM. Daarnaast is er nog een kwestie van klanten met een B2B-overeenkomst. Zij hebben inzage in de OneNotes, waardoor deze niet gemigreerd zijn. Voor deze klanten kan een oplossing gezocht worden via mogelijke licenties op RDM, een piste die verder onderzocht kan worden.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Effectieve documentatie is cruciaal voor IT-infrastructuurteams. Veel organisaties kampen echter met verspreide, inconsistente en verouderde documentatie, wat leidt tot inefficiëntie en kennisverlies (Abbas et al., [2022](#)).

A.1.1. Kadering en doelgroep

Dit onderzoek situeert zich binnen knowledge management en documentatiebeheer voor IT-infrastructuurteams, specifiek toegespitst op het Modern Workplace Infrastructure-team van ESC bv. De doelgroep omvat system administrators, infrastructuur engineers en IT-managers die verantwoordelijk zijn voor kennisbeheer.

A.1.2. Probleemstelling

Bij ESC bv is de documentatie van het Modern Workplace Infrastructure-team verspreid over verschillende platformen, inconsistent en vaak niet up-to-date. Dit resulteert in problemen bij onboarding van nieuwe teamleden, tijdverlies bij het zoeken naar informatie, en inefficiënte kennisoverdracht (Walker, [2007](#)).

A.1.3. Onderzoeksvraag

De centrale onderzoeksvraag luidt: **Hoe kan verspreide, inconsistente en vaak verouderde documentatie bij ESC bv geautomatiseerd, gecentraliseerd en gesynchroniseerd worden binnen het team van Modern Workplace Infrastructure, zodat de efficiëntie bij onboarding, kennisdeling en operationele opvol-**

ging geoptimaliseerd kan worden?

Deze hoofdvraag wordt ondersteund door volgende deelvragen:

Probleemdomein:

- Hoe kan binnen een termijn van 2 weken inconsistente documentatie binnen het team van Modern Workplace Infrastructure geïdentificeerd worden en kan dit proces geautomatiseerd worden?
- Hoe kunnen de noden van de teamleden binnen een termijn van 4 weken in kaart gebracht worden?
- Welke gecentraliseerde tools worden momenteel gebruikt en hoe kan binnen een termijn van 12 weken een gegronde keuze gemaakt worden voor één systeem?

Oplossingsdomein:

- Hoe kan binnen de stageperiode bestaande documentatie automatisch geanalyseerd en gestructureerd worden, zodat hiaten en inconsistenties geïdentificeerd kunnen worden?
- Hoe kan de overstap gemaakt worden naar een gecentraliseerde en uniforme documentatieomgeving, en hoe kan dit blijvend onderhouden worden?

A.1.4. Onderzoeksdoelstelling

Het concrete eindresultaat is een proof-of-concept van een gecentraliseerde documentatieomgeving met volgende deliverables:

- Analyse van de huidige documentatiesituatie met identificatie van hiaten
- Onderbouwd advies voor een documentatieplatform (SharePoint, Confluence)
- Python/PowerShell scripts voor inventarisatie, metadata-extractie en duplicaatdetectie
- Git repository met versiebeheer voor documentatie
- CI/CD pipeline voor geautomatiseerde validatie en deployment
- Synchronisatie-scripts tussen Git en het gekozen platform
- Validation framework met automated checks (formatting, metadata, broken links)
- Logging en monitoring voor alle geautomatiseerde processen
- Templates, richtlijnen en implementatieplan

De bachelorproef is succesvol wanneer de proof-of-concept aantoonbaar de efficiëntie verhoogt en positieve feedback krijgt van het team.

A.2. Literatuurstudie

A.2.1. Knowledge Management en documentatiefragmentatie

Abbas et al. (2022) tonen aan dat organisaties met verspreide en inconsistente documentatie significant lagere productiviteit ervaren. Kritieke succesfactoren zijn: centralisatie, uniforme structuur, regelmatige updates en toegankelijkheid.

Bento et al. (2020) onderzoeken organisatorische silo's en hun impact op kennisdeling. Verspreide documentatie is vaak het gevolg van verschillende teams die eigen systemen hanteren. Het onderzoek benadrukt dat centralisatie gecombineerd moet worden met cultuurverandering en continu onderhoud.

A.2.2. Best practices en technologieën

Walker (2007) beschrijft best practices voor documentconsolidatie: grondige inventarisatie vooraf, duidelijke taxonomie, metadata-standaarden, versiebeheer en automatisering voor duplicaatdetectie.

Voor gecentraliseerde documentatie komen verschillende platformen in aanmerking. SharePoint Online integreert goed met Microsoft 365 en ondersteunt automatisering via Power Automate. Confluence (Atlassian) biedt flexibele mogelijkheden voor technische documentatie. OneNote is meer informeel en mist enterprise governance-features.

A.2.3. DevOps voor documentatie

"Docs as Code" behandelt documentatie met dezelfde rigor als broncode: Git versiebeheer, automated testing en CI/CD. CI/CD pipelines kunnen validaties uitvoeren zoals Markdown linting, broken links detection en metadata checks via tools zoals GitHub Actions of Azure DevOps.

Python en PowerShell zijn geschikt voor documentatie-automation. Python biedt libraries voor API-integratie en data-analyse, PowerShell integreert met Microsoft Graph API. Monitoring via Azure Application Insights of ELK stack zorgt voor traceerbaarheid.

A.2.4. Kennishiaat

Er bestaat weinig onderzoek specifiek gericht op Modern Workplace Infrastructure-teams bij middelgrote IT-dienstverleners. Ook de toepassing van DevOps-praktijken (CI/CD, automated testing) voor documentatiebeheer is onderbelicht. Dit onderzoek ontwikkelt een geïntegreerde, praktijkgerichte oplossing met nadruk op technische automatisering.

A.3. Methodologie

Dit onderzoek volgt een praktijkgerichte aanpak die elementen combineert van requirements-analyse, vergelijkende studie en de ontwikkeling van een proof-of-

concept. De methodologie is opgedeeld in vijf fasen binnen een periode van 12 weken, elk met specifieke technieken, deliverables en tijdschatting.

A.3.1. Fase 1: Inventarisatie en audit (1,5 weken)

Inventarisatie van alle locaties waar documentatie opgeslagen is (SharePoint, OneDrive, Teams, lokale drives). Ontwikkeling van Python-scripts met Microsoft Graph API voor geautomatiseerd scannen en extractie van metadata. Detectie van duplicaten via file hashing (SHA-256) en data-analyse met pandas. SQLite database voor opslag, unit tests met pytest.

Deliverables: Inventarisatierapport, Python/PowerShell scripts, database met metadata.

A.3.2. Fase 2: Requirements-analyse (2,5 weken)

Semi-gestructureerde interviews met teamleden, observatie van documentatiegebruik, analyse van use cases. Identificatie van must-have features.

Deliverables: Requirements-document met user stories en geprioriteerde features.

A.3.3. Fase 3: Vergelijkende studie (2 weken)

Vergelijkingsmatrix voor SharePoint Online, Confluence en alternatieven op basis van criteria zoals gebruiksvriendelijkheid, integratie, automatiseringsmogelijkheden en kosten. Praktische evaluatie via proof-of-concept voor top kandidaten.

Deliverables: Vergelijkend rapport met onderbouwde platformkeuze.

A.3.4. Fase 4: Ontwerp en ontwikkeling (4 weken)

Ontwerp van een uniforme taxonomie, documentatietemplates en metadata-schema's. Implementatie van:

- Git repository met branching strategy voor versiebeheer
- CI/CD pipeline (GitHub Actions/Azure DevOps) voor validatie en deployment
- PowerShell/Python synchronisatie-scripts tussen Git en SharePoint
- Validation framework: automated checks op formatting, metadata, broken links
- Logging en monitoring (Azure Application Insights)
- Scheduled automation via Azure Functions of PowerShell tasks

Deliverables: Werkende proof-of-concept met Git repository, CI/CD pipeline, automation scripts met unit tests, monitoring dashboard.

A.3.5. Fase 5: Evaluatie en implementatieplan (2 weken)

Usability testing met teamleden, feedback-sessies, KPI-metingen. Opstellen van onderhoudsrichtlijnen, implementatieplan en trainingsmateriaal.

Deliverables: Evaluatierapport, implementatieplan, onderhoudsrichtlijnen, trainingsmateriaal.

A.3.6. Tijdsplanning overzicht

- Week 1-2: Fase 1 - Inventarisatie en audit
- Week 3-5: Fase 2 - Requirements-analyse
- Week 6-7: Fase 3 - Vergelijkende studie
- Week 8-11: Fase 4 - Ontwerp en ontwikkeling
- Week 12: Fase 5 - Evaluatie en implementatieplan

Deze aanpak garandeert voldoende technische diepgang door de ontwikkeling van scripts, automatisering en een werkende proof-of-concept, en zorgt tegelijk voor een praktisch toepasbaar resultaat dat direct waarde biedt aan ESC bv.

A.4. Verwacht resultaat, conclusie**A.4.1. Verwachte bevindingen**

De inventarisatie zal naar verwachting 5-8 verschillende documentatielocaties identificeren met 30-40% duplicaten of verouderde versies. SharePoint Online zal waarschijnlijk als meest geschikte platform uit de vergelijkende studie komen gezien de integratie met Microsoft 365 en automatiseringsmogelijkheden.

A.4.2. Verwachte proof-of-concept

Het eindresultaat omvat een gecentraliseerde SharePoint site met hiërarchische structuur, templates, metadata-schema, Git repository, CI/CD pipeline voor validatie, synchronisatie-scripts, logging/monitoring dashboard en scheduled automation voor dagelijkse synchronisatie en periodieke audits.

A.4.3. Meetbare verbeteringen

Verwachte verbeteringen: 60-70% reductie zoektijd, 40-50% kortere onboarding, 90%+ documentkwaliteit binnen 6 maanden, 80%+ gebruikerstevredenheid. Technisch: 85%+ automation coverage, 99%+ sync success rate, 95%+ validation pass rate.

A.4.4. Meerwaarde

Dit onderzoek levert ESC bv een werkende oplossing die operationele efficiëntie verhoogt, kennisbehoud garandeert, en schaalbaarheid mogelijk maakt. De ont-

wikkelde scripts en automation zijn herbruikbaar voor andere teams. Het technische luik (Python/PowerShell, CI/CD, Git) toont voldoende diepgang voor toegepaste informatica.

Mogelijke afwijkingen: indien geen enkel platform volledig voldoet, kan een hybride oplossing overwogen worden. Bij complexere automatisering kan de scope aangepast worden naar semi-geautomatiseerde processen. De kernwaarde van centralisatie blijft behouden.

Bibliografie

- Abbas, Y., Martinetti, A., Rajabalinejad, M., Schubert, F., & van Dongen, L. A. M. (2022). Facilitating digital collaboration through knowledge management: a case study. *Journal of Knowledge Management*. Geraadpleegd op 3 november 2025, van <https://www.tandfonline.com/doi/full/10.1080/14778238.2022.2029597>
- Bento, F., Tagliabue, M., & Lorenzo, F. (2020). Organizational Silos: A Scoping Review Informed by a Behavioral Perspective on Systems and Networks. *Societies*, 10(3), 56. <https://doi.org/10.3390/soc10030056>
- Blobel, B., Nordberg, R., Davis, J. M., & Pharow, P. (2006). Modelling privilege management and access control. *International Journal of Medical Informatics*, 75(8), 597–623. <https://doi.org/10.1016/j.ijmedinf.2005.08.010>
- Devolutions. (2025). *Remote Desktop Manager*. Devolutions. Geraadpleegd op 20 februari 2026, van <https://devolutions.net/remote-desktop-manager/>
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture* (1st). Addison-Wesley.
- Garbis, J., & Chapman, J. W. (2021). Privileged Access Management. In *Zero Trust Security: An Enterprise Guide* (pp. 155–161). Springer. https://doi.org/10.1007/978-1-4842-6702-8_12
- Hay, T. (2025, augustus). *Confidential by design: How we're securing OneNote for the age of AI at Microsoft*. Microsoft. Geraadpleegd op 16 februari 2026, van <https://www.microsoft.com/insidetrack/blog/confidential-by-design-how-were-securing-onenote-for-the-age-of-ai-at-microsoft/>
- Microsoft. (2025a). *Microsoft Graph throttling guidance*. Microsoft. Geraadpleegd op 20 februari 2026, van <https://learn.microsoft.com/en-us/graph/throttling>
- Microsoft. (2025b). *Microsoft identity platform and the OAuth 2.0 device authorization grant flow*. Microsoft. Geraadpleegd op 20 februari 2026, van <https://learn.microsoft.com/en-us/entra/identity-platform/v2-oauth2-device-code>
- Microsoft. (2025c). *Register an application with the Microsoft identity platform*. Microsoft. Geraadpleegd op 20 februari 2026, van <https://learn.microsoft.com/en-us/entra/identity-platform/quickstart-register-app>
- Microsoft. (2025d). *Use the OneNote REST API*. Microsoft. Geraadpleegd op 20 februari 2026, van <https://learn.microsoft.com/en-us/graph/integrate-with-onenote>

- Sizov, N. (2024). *Securing Organizational Assets: A Comprehensive Analysis of Privileged Access Management* [Bachelor's thesis]. Haaga-Helia University of Applied Sciences. <https://www.theseus.fi/handle/10024/850550>
- Tomsu, M. (2024). Information Overload as a Problem for IT Specialist. In B. Katalinic (Red.), *Proceedings of the 35th DAAAM International Symposium* (pp. 177–182). DAAAM International. <https://doi.org/10.2507/35th.daaam.proceedings.023>
- Walker, P. (2007, 28 maart). *Best Practices in ECM Document Consolidation and Migration*. KMWorld. Geraadpleegd op 3 november 2025, van <https://www.kmworld.com/Articles/WhitePaper/Article/Best-Practices-in-ECM-Document-Consolidation-andMigration-35745.aspx>
- Waltermire, K., Conroy, T., Harriston, M., Irrechukwu, C., Krishnan, N., Memole-Doodson, J., Nkrumah, B., Perper, H., Prince, S., & Wynne, D. (2018, september). *Privileged Account Management for the Financial Services Sector* (NIST Special Publication Nr. 1800-18) (Draft). National Institute of Standards en Technology. <https://www.nccoe.nist.gov/sites/default/files/legacy-files/fs-pam-nist-sp1800-18-draft.pdf>